

Etude numérique de la stabilité longitudinale d'une coque planante

Projet Xilab



Auteur : Havret Anaëlle - Jabot Lisa
Formation : SeaTech - MOCA 2A
Encadrant : Ersoy Mehmet
Date : April 29, 2026

Abstract

This project aims to investigate the influence of the geometry of planing hulls on their dynamic pitch stability. The aim is to investigate the phenomenon of porpoising. To this end, we are modelling configurable hulls, ranging from a simple rectangular box model to more complex hulls. Numerical simulations are carried out using CFD with *OpenFOAM* and *Flow3D*, and are compared with theoretical calculations to validate the initial test cases.

The porpoising phenomenon is a self-sustaining pitch instability observed in the transition zone between the moving regime and the planing regime. Results from the interaction between the centre of gravity and the centre of pressure, with rapid variations in lift and drag. This study establishes a link between the geometric characteristics of the hull and the onset, amplitude and frequency of porpoising, providing useful insights for the design of more stable high-speed hulls.

Keywords: porpoising, planing hull, hydrodynamic lift, drag, pitch, CFD, *OpenFOAM*, *Flow3D*

Résumé

Ce projet vise à étudier l'influence de la géométrie des coques planantes sur leur stabilité dynamique en tangage. On cherche à y étudier le phénomène de marsouinage. On modélise alors des coques paramétrables, allant d'un modèle simple de boîte rectangulaire à des coques plus complexes. Les simulations numériques sont réalisées en CFD avec *OpenFOAM* et *Flow3D*, et sont confrontées à des calculs théoriques pour valider les premiers cas tests.

Le phénomène de marsouinage, instabilité auto-entretenu de tangage observée dans la zone de transition entre régime de déplacement et régime planant, résulte de l'interaction entre le centre de gravité et le centre de pression, avec des variations rapides de la portance et de la traînée. Cette étude permet de relier les caractéristiques géométriques de la coque à l'apparition, à l'amplitude et à la fréquence du marsouinage, fournissant des indications utiles pour la conception de coques rapides plus stables.

Mots-clés : marsouinage, coque planante, portance hydrodynamique, traînée, tangage, CFD, *OpenFOAM*, *Flow3D*

Remerciements

Nous tenons à adresser nos sincères remerciements à notre professeur encadrant, Mehmet Ersoy, pour son accompagnement, sa disponibilité et ses conseils tout au long de ce projet.

Nous remercions également la société Flow-3D ainsi que Julien Nugue pour son aide précieuse, ses explications techniques et son soutien dans l'utilisation des outils de simulation.

Nos remerciements vont aussi à l'entreprise Xilab, qui nous a proposé ce sujet intéressant et formateur, nous permettant de travailler sur un cas concret et enrichissant.

Enfin, nous exprimons notre gratitude aux informaticiens pour leur intervention efficace dans l'installation du logiciel, sans laquelle ce travail n'aurait pas pu être mené à bien.

Merci à tous pour votre contribution à la réussite de ce projet.

Table des matières

1	Introduction	4
1.1	Contexte et état de l'art	4
1.2	Objectif et organisation du rapport	5
2	Modélisation théorique	6
2.1	Etude hydrostatique	6
2.2	Etude hydrodynamique	7
2.3	Comparaison des géométries	13
2.3.1	Cas test cuboïde	13
2.3.2	Parallélépipède rectangle	14
2.3.3	Coque planante	16
2.3.4	Cas test : planing hull	20
3	Modélisation numérique	20
3.1	Cas test cuboïde	21
3.1.1	Etude OpenFOAM	21
3.1.2	Etude Flow3D	26
3.1.3	Comparaison des logiciels	32
3.2	Cas test : planing hull	34
3.2.1	Paramètres des simulations	34
3.2.2	Résultats et comparaison	35
3.3	Parallélépipède rectangle	36
3.4	Coque planante	38
3.4.1	Etude de la convergence	38
3.4.2	Flow 3D	38
3.5	Comparaison et discussion des résultats	41
4	Conclusion	45
A	Annexe	47

1 Introduction

Le phénomène de marsouinage (*porpoising* en anglais) tire son nom du comportement des dauphins et des marsouins, qui se déplacent en effectuant des mouvements oscillatoires hors de l'eau. Par analogie, ce terme désigne, en hydrodynamique navale, une instabilité dynamique caractérisée par des oscillations en tangage d'une coque planante.

Lorsque la vitesse du bateau augmente, la portance hydrodynamique devient prépondérante et tend à soulever la coque hors de l'eau. Dans certaines conditions, ce phénomène ne se stabilise pas : le bateau entre alors dans un régime instable où il se soulève, puis retombe de manière répétée. Ce mouvement oscillatoire auto-entretenu correspond au marsouinage.

Dans le cadre de cette étude, nous nous intéressons particulièrement à ce phénomène. L'objectif est d'observer et d'analyser les oscillations en tangage du bateau, caractérisées par une succession de phases de montée (augmentation de l'assiette) et de retombée sous l'effet combiné de la gravité et des variations de portance.

L'étude de ce phénomène est essentielle pour la conception des coques rapides, car il affecte à la fois la stabilité, le confort et la sécurité du navire.

1.1 Contexte et état de l'art

Les coques planantes atteignent des vitesses élevées grâce à la génération de portance hydrodynamique, ce qui permet de réduire significativement la surface mouillée et donc la résistance à l'avancement. Le régime de planage, caractérisé par une domination des forces de portance sur la poussée hydrostatique, a été formalisé par [12], dont les travaux constituent encore aujourd'hui une référence pour la prédiction de l'assiette, de la traînée et des efforts hydrodynamiques.

Les approches théoriques classiques ont été progressivement complétées par des études expérimentales et numériques permettant de mieux prendre en compte la complexité des écoulements réels. En particulier, l'utilisation de la mécanique des fluides numérique (CFD) permet d'intégrer les effets visqueux, turbulents et tridimensionnels, offrant ainsi une description plus fidèle des phénomènes hydrodynamiques [7, 9, 8]. Ces outils numériques permettent notamment de capturer les interactions fluide-structure, la formation des vagues et les variations de portance avec une grande précision.

De nombreuses études ont démontré la capacité de la CFD à reproduire des configurations réalistes, y compris dans des conditions spécifiques telles que les eaux restreintes ou les interactions avec les parois [3]. Les logiciels comme *OpenFOAM* sont largement utilisés pour l'étude des structures flottantes et des phénomènes de surface libre [5, 6, 11], tandis que des travaux récents proposent des analyses comparatives et des validations expérimentales des modèles numériques.

Par ailleurs, des travaux de synthèse récents mettent en évidence le rôle déterminant de la géométrie de coque (angle de deadrise, largeur, longueur) ainsi que des paramètres opérationnels (vitesse, masse, position du centre de gravité) dans les performances globales des coques

planantes [4]. Ces paramètres influencent directement la portance, la traînée et la stabilité du navire.

Les bases théoriques de l'architecture navale, notamment présentées dans les ouvrages de référence [10] et les notes de cours de E. Dufour [2], permettent de relier ces phénomènes hydrodynamiques aux notions fondamentales de stabilité, d'équilibre et de comportement dynamique des navires. Elles fournissent un cadre essentiel pour interpréter les résultats issus des simulations numériques.

Cependant, la stabilité dynamique des coques planantes reste un enjeu majeur, en particulier à grande vitesse. Le phénomène de marsouinage (*porpoising*) correspond à une instabilité oscillatoire en tangage résultant d'un couplage instable entre la portance hydrodynamique et l'assiette du navire. Il apparaît généralement au-delà d'une vitesse critique et dépend fortement de la position du centre de gravité, de la géométrie de la coque et des conditions d'exploitation [15].

Des travaux expérimentaux et théoriques, notamment ceux de Blount et Codega [1], ont mis en évidence la sensibilité de ces instabilités à la répartition des masses et aux paramètres de navigation. Ces études montrent que de faibles variations de configuration peuvent entraîner des changements significatifs du comportement dynamique, rendant nécessaire une analyse fine et couplée des phénomènes hydrodynamiques et dynamiques.

Ainsi, l'étude du marsouinage et plus généralement de la stabilité dynamique des coques planantes nécessite une approche combinant modèles théoriques, simulations numériques et validation expérimentale afin de mieux comprendre et prédire ces phénomènes complexes.

1.2 Objectif et organisation du rapport

Dans ce rapport, nous nous intéressons à l'influence de la géométrie des coques planantes sur leur stabilité dynamique en tangage. L'objectif est de simuler numériquement la stabilité de plusieurs coques à l'aide de logiciels CFD, puis de confronter les résultats obtenus à des calculs théoriques.

Les logiciels utilisés sont *OpenFOAM* et *Flow3D Hydro*, le but étant également de valider la pertinence et la justesse de ce second logiciel plus récent.

Le rapport est organisé comme suit. Tout d'abord, la Section 2 présente l'étude d'un cube réalisée pour un premier cas test à géométrie simple, puis d'un parallélépipède rectangle et de deux coques. Enfin, la Section 3 regroupe les résultats obtenus sous deux logiciels différents : *OpenFOAM* et *Flow3D* ainsi que leur comparaison et analyse pour chacun des cas.

2 Modélisation théorique

Etudions la statique et la dynamique d'une structure flottante afin de déterminer sa stabilité et de caractériser un régime planant efficace.

2.1 Etude hydrostatique

Dans cette partie, nous allons étudier la stabilité d'une structure flottante [2]. Afin de déterminer la stabilité, nous devons calculer la hauteur métacentrique (GM). En effet si celle-ci est positive, alors la structure est stable pour une certaine variation d'angle. En revanche si elle est négative, la structure "se retourne" lorsqu'on lui donne un angle de rotation.

On définit plusieurs types de mouvements pour une structure flottante (par ex un bateau) :

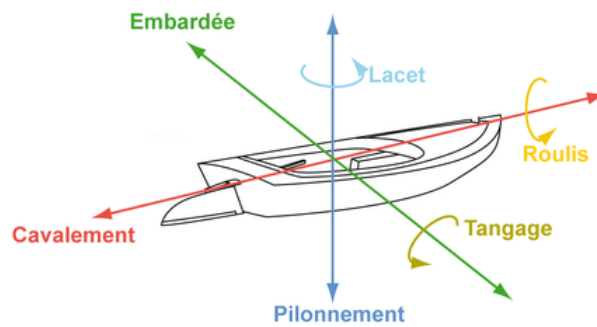


FIGURE 1 : Degrés de liberté d'un bateau

Dans le cadre de notre étude, nous souhaitons étudier la stabilité longitudinale (GM en tangage), mais nous allons également étudier la stabilité en roulis (Figure 1).

D'après la seconde loi de Newton, on a :

$$\sum \vec{F}_{ext} = \vec{0}$$

$$\Leftrightarrow \vec{P} + \vec{\Pi}_a = \vec{0}$$

Avec $\vec{P}$ le poids et $\vec{\Pi}_a$ la poussée d'Archimède les forces qui s'appliquent sur la structure (Figure 2).

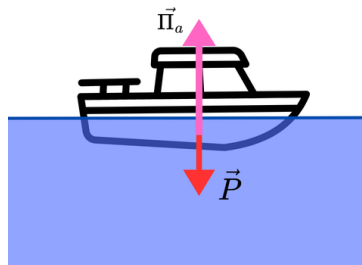


FIGURE 2 : Schéma des forces

On a donc selon l'axe z :

$$-mg + \rho_w V_i g = 0$$

$$\Leftrightarrow m = \rho V_i$$

Avec m la masse de la boîte, ρ_w la masse volumique de l'eau de salée et V_i le volume immergé de la boîte également. Sachant que la structure est enfoncée d'une hauteur d dans l'eau, on a :

$$m = \rho_w \cdot d \cdot L^2$$

$$\Leftrightarrow d = \frac{m}{\rho_w \cdot L \cdot B}$$

Le volume immergé est donc de $V_i = d \cdot L^2 = 0.013 \text{ m}^3$.

La hauteur métacentrique est définie de telle sorte que :

$$GM = BM + KB - KG$$

Où le point B se situe au niveau du centre de flottabilité de la structure, le point G est le centre de gravité, K est au niveau de la quille tout en bas, et M est le métacentre.

On a les formules :

$$\begin{cases} BM = \frac{I}{V_i} \\ KB = \frac{d}{2} \\ KG = \frac{L}{2} \end{cases}$$

Où I est l'inertie quadratique selon l'axe de rotation de la structure, en m^4 .

2.2 Etude hydrodynamique

Équilibre des forces en régime permanent :

Dorénavant, nous allons étudier la dynamique du bateau. On suppose que le bateau avance à vitesse constante $\vec{u}$. Lorsque le bateau avance, trois forces s'ajoutent au bilan des forces : la force de traînée (F_d), de portance (F_l) et celle du moteur du bateau (F_m). Nous avons ainsi le schéma des forces suivant (Figure 3) :

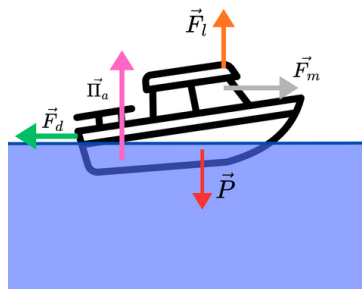


FIGURE 3 : Schéma des forces

D'après la 2^{nde} loi de Newton :

$$\vec{F}_l + \vec{F}_d + \vec{P} + \vec{F}_m + \vec{\Pi}_a = \vec{0}$$

Avec $F_l = \frac{1}{2}C_l\rho Su^2$ et $F_d = \frac{1}{2}C_d\rho Su^2$.

En projetant sur les axes $\vec{e}_x$ et $\vec{e}_y$ et en supposant que le bateau reste à plat (angle $\theta = 0^\circ$), on a :

$$\begin{cases} F_d = F_m \\ F_l + P + \Pi_a = 0 \end{cases}$$

Le poids étant toujours constant, la force de portance et d'Archimède doivent se compenser. Si la force de portance augmente avec la vitesse, la poussée d'Archimède diminue en réduisant le volume immergé. C'est ainsi que le bateau tangue d'un certain angle.

Équation du mouvement d'un bateau :

Le comportement dynamique d'un bateau peut être décrit par sa position $x_i(t)$. Comme tout système mécanique, il obéit à la seconde loi de Newton :

$$(M + M_a) \ddot{x}_i(t) + B\dot{x}_i(t) + Kx_i(t) = F_{\text{ext}}^{(i)}(t) \quad (1)$$

où :

- M : matrice de masse du navire
- M_a : matrice de masse ajoutée
- B : matrice d'amortissement
- K : matrice de raideur hydrostatique
- $F_{\text{ext}}^{(i)}$: forces extérieures

Pour le mouvement de rotation (notamment le tangage), l'équation des moments s'écrit :

$$(I_{yy} + I_a) \ddot{\theta}(t) + B_\theta \dot{\theta}(t) + K_\theta \theta(t) = M_{\text{ext}}(t) \quad (2)$$

où I_{yy} est le moment d'inertie du navire, I_a l'inertie ajoutée, $\theta(t)$ l'angle de tangage, et $M_{\text{ext}}(t)$ le moment exciteur dû aux vagues.

Dans le cas couplé heavepitch, les équations peuvent s'écrire sous forme matricielle :

$$\begin{cases} (M + M_a) \ddot{z} + B_{33} \dot{z} + K_{33} z + K_{35} \theta = F_z \\ (I_{yy} + I_a) \ddot{\theta} + B_{55} \dot{\theta} + K_{55} \theta + K_{53} z = M_y \end{cases} \quad (3)$$

La raideur hydrostatique en heave est donnée par :

$$K_{33} = \rho g A_{wp} \quad (4)$$

où A_{wp} est la surface de flottaison. Elle traduit la capacité du navire à retrouver sa position d'équilibre vertical.

La raideur en pitch dépend de la répartition de la surface de flottaison et s'écrit sous forme intégrale :

$$K_{55} = \rho g \int x^2 dA \quad (5)$$

Enfin, le phénomène de marsouinage correspond à une instabilité dynamique du système couplé heavepitch, caractérisée par une amplification auto-entretenu des oscillations verticales et angulaires à haute vitesse, lorsque l'amortissement devient insuffisant pour stabiliser la réponse du navire face à l'excitation des vagues.

Masse et masse ajoutée :

La masse du navire est donnée par :

$$M = \begin{pmatrix} m & 0 & 0 & 0 & 0 & 0 \\ 0 & m & 0 & 0 & 0 & 0 \\ 0 & 0 & m & 0 & 0 & 0 \\ 0 & 0 & 0 & I_{xx} & 0 & 0 \\ 0 & 0 & 0 & 0 & I_{yy} & 0 \\ 0 & 0 & 0 & 0 & 0 & I_{zz} \end{pmatrix} \quad (6)$$

La matrice de masse ajoutée :

$$M_a = \begin{pmatrix} m_{axx} & \cdots & m_{ax\phi} \\ \vdots & \ddots & \vdots \\ m_{a\phi x} & \cdots & m_{a\phi\phi} \end{pmatrix} \quad (7)$$

Une approximation est :

$$M_a^{(i,j)} = \rho_{\text{fluide}} C_a^{(i,j)} V_R^{(i,j)} \quad (8)$$

Avec ρ_{fluide} la masse volumique du fluide, $C_a^{(i,j)}$ les coefficients de masse ajoutée propres à la géométrie de l'objet flottant et $V_R^{(i,j)}$ le volume de référence de l'objet.

Comparaison des grandeurs hydrodynamiques :

On considère les forces hydrodynamiques extraites des simulations CFD :

- F_l : force selon l'axe longitudinal (résistance à l'avancement)
- F_d : force verticale (portance hydrodynamique)

Afin de caractériser le régime d'écoulement et les performances hydrodynamiques de la coque, plusieurs grandeurs adimensionnelles sont introduites. Le coefficient de portance et le coefficient de traînée permettent de comparer les forces hydrodynamiques indépendamment des conditions d'écoulement. Ils traduisent respectivement la capacité de la coque à générer de la portance et la résistance à l'avancement.

Coefficient de portance :

$$C_L = \frac{F_l}{\frac{1}{2}\rho S u^2}$$

Coefficient de traînée :

$$C_D = \frac{F_d}{\frac{1}{2}\rho S u^2}$$

Surface de référence :

Dans cette étude, la surface de référence est définie comme :

$$S = L \times B$$

où :

- L : longueur de la coque
- B : largeur maximale

Ratio portance / traînée :

Le rapport portance/traînée constitue un indicateur de performance. Un ratio faible correspond à un régime de déplacement dominé par la traînée, tandis qu'un ratio élevé traduit un régime planant efficace, dans lequel la portance hydrodynamique permet de réduire la surface mouillée et donc les pertes énergétiques.

$$\frac{F_l}{F_d} = \frac{C_L}{C_D}$$

- $\frac{C_L}{C_D} \ll 1$: régime inefficace
- $\frac{C_L}{C_D} > 1$: régime planant efficace

Nombre de Froude et régime planant :

On a

$$F_n = \frac{u}{\sqrt{g L}}$$

Avec :

- u : vitesse
- $g = 9.81 \text{ m/s}^2$
- L : longueur caractéristique

Le nombre de Froude, défini à partir de la vitesse et d'une longueur caractéristique, permet de situer le régime d'écoulement. Pour des valeurs faibles, l'écoulement est dominé par les effets gravitaires et le bateau évolue en régime de déplacement. À l'inverse, lorsque le nombre de Froude devient élevé, les effets inertiels dominent et la coque entre en régime planant.

- $F_n < 1$: régime de déplacement
- $F_n \gg 1$: régime planant

Strip theory - Découpe en tranches transversales :

La **strip theory** (méthode des bandes) consiste à décomposer la coque en $N_x = 80$ tranches transversales d'épaisseur dx . Pour chaque abscisse x_i , l'intersection exacte du maillage avec le plan $X = x_i$ est calculée par détection des passages d'arêtes :

$$\mathbf{p} = \mathbf{a} + t(\mathbf{b} - \mathbf{a}), \quad t = \frac{x_i - a_X}{b_X - a_X}, \quad t \in [0, 1] \quad (9)$$

Les points d'intersection forment le contour de la section immergée, à partir duquel on extrait :

- la largeur locale $B(x_i)$: étendue transversale (axe Y) ;
- le tirant local $T(x_i)$: profondeur de la section (axe Z) ;
- l'aire de section $A(x_i)$: calculée par le polygone Shapely issu de **polygonize**.

Le volume strip est alors :

$$\nabla_{\text{strip}} = \int_L A(x) dx \approx \sum_{i=1}^{N_x} A(x_i) \Delta x_i = \text{m}^3 \quad (10)$$

Masses ajoutées — Formules de Lewis :

Les coefficients de masse ajoutée sont obtenus à partir des solutions potentielles pour des corps cylindriques en fluide parfait. En particulier, la masse ajoutée d'un cylindre peut être interprétée comme la masse d'un volume équivalent de fluide accéléré [13, p. 6-10]. Cette approche conduit aux expressions utilisées dans la strip theory.

Heave — mouvement vertical (a_{33}) :

$$a_{33}(x) = \frac{\rho \pi}{2} \left(\frac{B(x)}{2} \right)^2 \quad (11)$$

La demi-largeur $B(x)/2$ est le rayon du demi-cercle équivalent. Cette expression est le résultat exact pour un cylindre circulaire oscillant verticalement en fluide parfait non visqueux.

Sway - mouvement latéral (a_{22}) :

$$a_{22}(x) = \frac{\rho \pi}{2} T(x)^2 \quad (12)$$

Roll - moment de masse ajoutée (a_{44}) :

$$a_{44}(x) = \frac{\rho \pi}{8} \left(\frac{B(x)}{2} \right)^2 \frac{T(x)^2}{4} \quad (13)$$

Surge (m_{11}) - approximation empirique :

Le mode de surge (oscillation axiale) n'est pas bien capté par une approche 2D. On utilise la relation empirique classique :

$$m_{11} \approx 0.07 \rho \nabla_{\text{strip}} \quad (14)$$

Intégration longitudinale :

Les masses ajoutées globales sont obtenues par integration trapézoidale :

$$m_{ij} = \int_0^L a_{ij}(x) dx \approx \sum_{i=1}^{N_x} a_{ij}(x_i) \Delta x_i \quad (15)$$

Inerties quadratiques ajoutées :

L'inertie de masse ajoutée en tangage est le moment du second ordre de $a_{33}(x)$ par rapport au centre de gravité de sa distribution :

$$I_{55} = \int_0^L a_{33}(x) (x - x_G)^2 dx \quad (16)$$

avec le centre x_G défini par :

$$x_G = \frac{\int_0^L a_{33}(x) x dx}{\int_0^L a_{33}(x) dx} \quad (17)$$

En roulis :

$$I_{44} = \int_0^L a_{44}(x) dx \quad (18)$$

2.3 Comparaison des géométries

Dans cette partie, nous allons étudier plus précisément la stabilité de différentes géométries afin de les comparer. Les structures étudiées sont : un cube, un parallélépipède rectangle, une coque planante dessinée sur le logiciel *Rhino3D* ainsi qu'une coque planante extraite d'*OpenFOAM*.

2.3.1 Cas test cuboïde

Dans cette section, nous étudions théoriquement la stabilité statique et dynamique d'un cas test. Le cas étudié ici est un cube fermé de côtés $L = 0.3$ m (Figure 4).

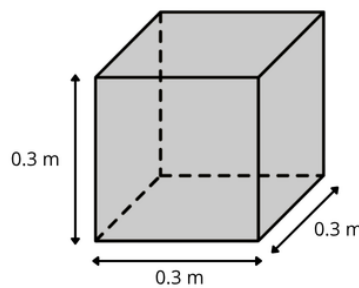


FIGURE 4 : Schéma du cas test

Le cube possède une masse volumique $\rho = 500$ kg/m³ et son volume est égal à $V = L^3 = 0.027$ m³. La masse est uniformément répartie dans le cube et vaut : $m = \rho V = 13.5$ kg.

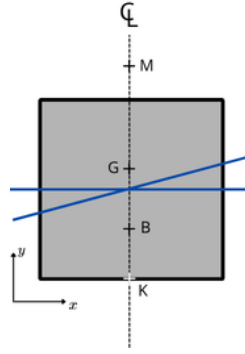


FIGURE 5 : Schéma points remarquables en roulis et en tangage

Comme

$$d = \frac{m}{\rho_w \cdot L^2}$$

Et que $\rho_w = 1025 \text{ kg/m}^3$ et $L = 0.3 \text{ m}$, alors on trouve $d = 0.146 \text{ m}$. Le volume immergé est donc de $V_i = d \cdot L^2 = 0.013 \text{ m}^3$.

Calculons à présent la hauteur métacentrique. On sait que :

$$GM = BM + KB - KG$$

$$\text{Donc : } \begin{cases} BM = \frac{I}{V_i} \\ KB = \frac{d}{2} = 0.073 \text{ m} \\ KG = \frac{L}{2} = 0.15 \text{ m} \end{cases}$$

Ici, nous avons un cube donc l'inertie quadratique est la même selon les axes x et z , on a donc :

$$I_x = I_z = \frac{L^4}{12} = 0.000675 \text{ m}^4$$

Stabilité (selon x – roulis) : $BM = \frac{I_x}{V_i} = 0.052 \text{ m}$, soit $GM = -0.025 \text{ m}$.

Stabilité longitudinale (selon z – tangage) : $BM = \frac{I_z}{V_i} = 0.052 \text{ m}$, soit $GM = -0.025 \text{ m}$.

Conclusion : La hauteur métacentrique (GM) est négative donc le cube n'est pas stable et a tendance à se retourner à l'approche d'une source d'instabilité (vague etc.).

2.3.2 Parallélépipède rectangle

Dans cette section, nous étudions théoriquement la stabilité statique et dynamique d'une boîte rectangulaire fermée de côté $L = 10 \text{ m}$, $B = 3 \text{ m}$, $H = 2 \text{ m}$ avec un volume de $V = 60 \text{ m}^3$ (Figure 6). On a créé cette boîte afin qu'elle fasse les mêmes dimensions que la coque planante. Elle a la même longueur, largeur et hauteur mais avec volume différent.

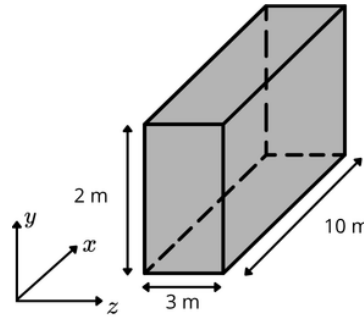
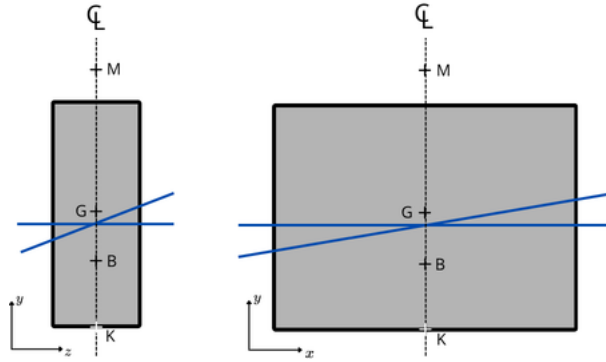


FIGURE 6 : Schéma de la boîte

La boîte possède une masse volumique : $\rho = 140 \text{ kg/m}^3$. On impose une masse uniformément répartie : $m = 8.4$ tonnes.

Son volume immergé est

$$V_i = \frac{m}{\rho_w} = 8.2 \text{ m}^3$$

FIGURE 7 : Schéma points remarquables en roulis (*gauche*) et en tangage (*droite*)

Son centre de gravité se situe en son centre : $KG = 1 \text{ m}$.

Son centre de carène : $KB = \frac{d}{2} = 0.25 \text{ m}$

En roulis :

$$I_{roll} = \frac{LB^3}{12} = \frac{10 \times 3^3}{12} = 22.5$$

$$BM_{roll} = \frac{I_{roll}}{V} = \frac{22.5}{8.2} \approx 2.74 \text{ m}$$

$$GM_{roll} = KB + BM_{roll} - KG = 0.25 + 2.74 - 1 \approx 2.0 \text{ m}$$

En tangage :

$$I_{pitch} = \frac{BL^3}{12} = \frac{3 \times 10^3}{12} = 250$$

$$BM_{pitch} = \frac{I_{pitch}}{V} = \frac{250}{8.2} \approx 30.5 \text{ m}$$

$$GM_{pitch} = KB + BM_{pitch} - KG = 0.25 + 30.5 - 1 \approx 29.8 \text{ m}$$

Conclusion : Le GM est positif donc la boîte est stable en tangage ainsi qu'en roulis.

Masse ajoutée :

Estimons à présent sa masse ajoutée :

$$m_a = C_a \times \rho_w \times v_R \quad (19)$$

La table de la DNVGL-RP-C205 [14] donne des coefficients de masse ajoutée pour un parallélépipède rectangle en fonction du rapport L/B pour un mouvement vertical (selon z) : on a $L = 10$ m et $B = 3$ m soit $H/B = 3.33$ et on a ainsi $C_a = 0.24$, avec $v_R = B^2 h = 90 \text{ m}^3$.

Ainsi obtient :

$$m_a \approx 2.2 \text{ tonnes} \quad (20)$$

Ainsi, la masse ajoutée est du même ordre de grandeur que la masse propre de la structure. La masse effective du système devient donc :

$$m_{\text{eff}} = m + m_a \approx 8.4 + 2.2 = 10.6 \text{ tonnes} \quad (21)$$

Ce résultat montre que les effets d'inertie du fluide sont prépondérants dans la dynamique du système, et doivent impérativement être pris en compte pour l'étude des phénomènes instationnaires tels que le tangage et le marsouinage.

2.3.3 Coque planante

Dans cette section, nous étudions la stabilité statique et dynamique d'une coque planante.

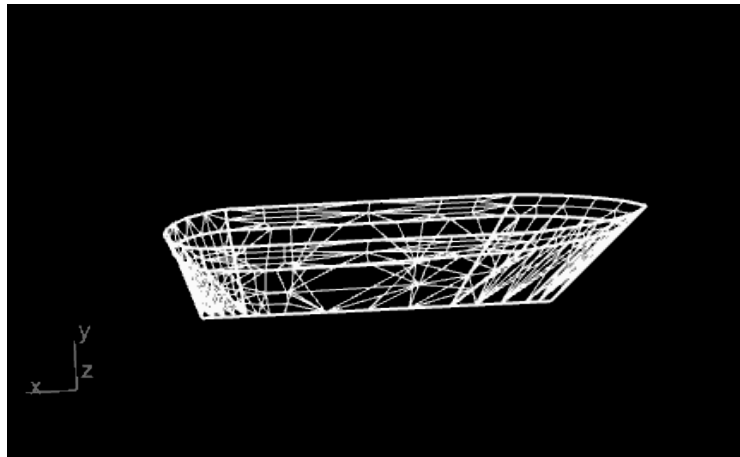


FIGURE 8 : Dessin d'une coque planante sur *Rhino3D*

Une coque planante de bateau a été modélisée à l'aide du logiciel *Rhinoceros 3D* afin de servir de géométrie de référence pour les simulations hydrodynamiques. Le modèle possède les dimensions suivantes : une longueur $L = 10$ m, une largeur $B = 3$ m et une hauteur totale de 2 m. Le volume total de la coque est d'environ 15 m^3 . La géométrie est caractérisée par un

angle de deadrise de 45° , correspondant à une coque en V typique des embarcations planantes, permettant d'étudier son comportement en régime de navigation.

Pour faciliter le dessin sur ce logiciel, nous avons créé un modèle stl deux fois plus grand que nous avons ensuite réduit sur flow3d.

Les conditions de chargement du bateau sont définies à partir de la masse maximale avant que celui-ci ne coule : $M_{\max} = 16$ tonnes, tandis que la masse imposée pour les calculs est $m = 8.2$ tonnes. Cela conduit à une masse volumique équivalente de l'ordre de $\rho_{eq} \approx 290 \text{ kg/m}^3$, utilisée pour caractériser le comportement global de flottabilité du modèle. Le volume immergé de référence est $V_i = 8.0 \text{ m}^3$, obtenu par le théorème de la divergence appliqué au maillage STL de la carène immergée à l'échelle réelle.

Les positions verticales caractéristiques du navire sont définies par le centre de carène et le centre de gravité. On obtient un centre de carène situé à $KB = 1.04 \text{ m}$ au-dessus de la quille, tandis que le centre de gravité est placé à $KG = 1.15 \text{ m}$.

La hauteur métacentrique est calculée séparément pour le roulis et le tangage :

$$GM = KB + BM - KG, \quad BM = \frac{I_{wl}}{V_i} \quad (22)$$

Le moment d'inertie quadratique I_{wl} est un moment d'inertie (homogène à m^4), analogue à celui utilisé en résistance des matériaux pour les sections de poutres. Il mesure la résistance de la surface de flottaison à une rotation :

$$I_{wl} = \iint_{S_{wl}} y^2 dA \quad (23)$$

où y est la distance à l'axe de rotation (axe longitudinal pour le roulis, axe transversal pour le tangage) et dA un élément de surface du plan d'eau et ne fait intervenir aucune masse.

Calcul numérique sur le polygone STL : La surface de flottaison est obtenue par intersection exacte du maillage STL avec le plan $Z = -1.0 \text{ m}$, produisant 108 segments qui forment le contour de la section. Le calcul de I_{wl} sur ce polygone est effectué par la formule de Shoelace généralisée :

$$I_{wl} = \frac{1}{12} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) (y_i^2 + y_i y_{i+1} + y_{i+1}^2) \right| - A_{wl} \bar{y}^2 \quad (24)$$

Cette formule intègre $\iint y^2 dA$ sur un polygone quelconque ; le terme $-A_{wl} \bar{y}^2$ est la correction de Steiner qui ramène le résultat à l'axe passant par le centroïde $\bar{y}$ de la section. La loi de similitude donne ensuite :

$$I_{wl,\text{réel}} = \lambda^4 \cdot I_{wl,\text{modèle}} \quad (\lambda^4 = 0.0625)$$

car I_{wl} est homogène à une longueur à la puissance 4. Les résultats sont :

Grandeur	Modèle STL	Facteur	Réel
Aire de flottaison A_{wl}	90.0 m ²	$\times \lambda^2$	22.5 m ²
I_{wl} roulis (axe long.)	212.17 m ⁴	$\times \lambda^4$	13.26 m ⁴
I_{wl} tangage (axe tran.)	2207.38 m ⁴	$\times \lambda^4$	137.96 m ⁴
Volume immergé V_i	64.0 m ³	$\times \lambda^3$	8.0 m ³

Roulis :

$$BM_\phi = \frac{13.26}{8.0} = 1.66 \text{ m} \quad (25)$$

$$GM_\phi = 1.04 + 1.66 - 1.15 = 1.55 \text{ m} > 0 \quad (26)$$

Tangage :

$$BM_\theta = \frac{137.96}{8.0} = 17.25 \text{ m} \quad (27)$$

$$GM_\theta = 1.04 + 17.25 - 1.15 = 17.14 \text{ m} > 0 \quad (28)$$

Conclusion : Les deux hauteurs métacentriques sont positives donc le navire est stable en roulis et en tangage.

Masses ajoutées et inerties quadratiques (échelle $\lambda = 1/2$) : Lorsqu'un corps se déplace dans un fluide, il entraîne avec lui une certaine quantité de fluide. Cette masse de fluide entraîné s'appelle masse ajoutée : elle s'additionne à la masse réelle du navire dans les équations du mouvement et augmente donc la résistance à l'accélération. Pour le tangage, l'analogue est une inertie ajoutée I_{55} (en kg·m²), qui s'additionne à l'inertie propre du navire.

Les masses ajoutées sont calculées par la strip theory de Lewis sur le maillage STL ($\lambda = 1/2$), avec le volume immergé $V_i = 8.0 \text{ m}^3$ comme référence pour le surge :

$$m_{11} \approx 0.07 \rho V_i = 0.07 \times 1000 \times 8.0 = 560 \text{ kg} \quad (29)$$

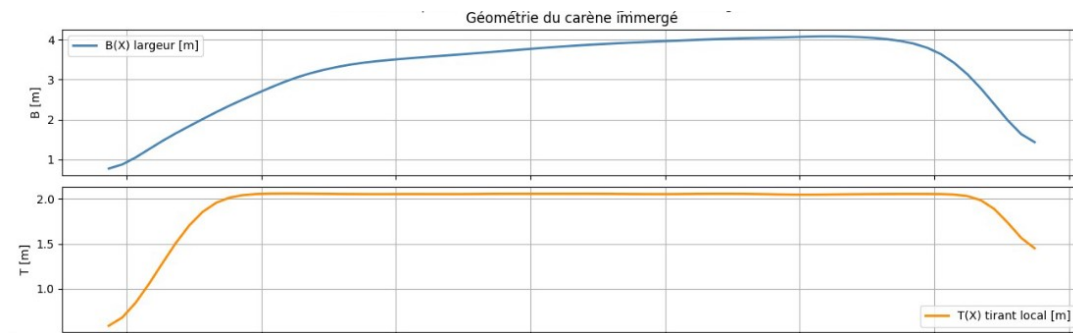


FIGURE 9 : Géométrie de la carène immergée

Les autres masses ajoutées sont obtenues par intégration des formules de Lewis et ramenées à l'échelle réelle par $m_{ij,\text{réel}} = \lambda^3 m_{ij,\text{mod}}$ et $I_{ij,\text{réel}} = \lambda^5 I_{ij,\text{mod}}$.

TABLE 1 : Masses ajoutées et inerties quadratiques à l'échelle réelle ($\lambda = 1/2$, $V_i = 8.0 \text{ m}^3$)

DDL	Mouvement	Symbole	Modèle STL	Réel
1	Surge	m_{11}	4 480 kg	560 kg
2	Sway	m_{22}	239 528 kg	29 941 kg
3	Heave(pillonement)	m_{33}	194 371 kg	24 296 kg
4	Roll (Roulis)	I_{44}	109 588 m^4	3 425 m^4
5	Pitch(Tangage)	I_{55}	3 894 028 $\text{kg}\cdot\text{m}^2$	121 688 $\text{kg}\cdot\text{m}^2$

Les masses effectives du système (navire + fluide ajouté) sont :

$$M_{11} = m + m_{11} = 8\,200 + 560 = 8\,760 \text{ kg} \quad (30)$$

$$M_{22} = m + m_{22} = 8\,200 + 29\,941 = 38\,141 \text{ kg} \quad (31)$$

$$M_{33} = m + m_{33} = 8\,200 + 24\,296 = 32\,496 \text{ kg} \quad (32)$$

La masse ajoutée en surge représente seulement 6.8 % de la masse du navire ($m_{11}/m \approx 0.07$). Ce faible couplage s'explique par la forme effilée de la proue et de la poupe de la coque planante : le fluide est peu accéléré lors d'un mouvement axial. La masse effective $M_{11} = 8\,760 \text{ kg}$ est donc quasi-identique à la masse réelle, et la résistance hydrodynamique en surge est dominée par la traînée de frottement plutôt que par l'inertie ajoutée.

L'inertie ajoutée en tangage est donc 1.8 fois supérieure à l'inertie propre du navire. Cela signifie que lors d'un mouvement de tangage, la colonne d'eau entraînée contribue bien davantage à la résistance au mouvement que la structure elle-même.

Raideur et période propre en tangage : K_{55} est le coefficient de raideur hydrostatique en tangage (couple de redressement par unité d'angle) :

$$C_{55} = \rho g V_i G M_\theta = 1000 \times 9.81 \times 8.0 \times 17.14 = 1\,345\,147 \text{ N}\cdot\text{m}/\text{rad} \quad (33)$$

K_{55} est calculé à partir de $G M_\theta$ lui-même issu de I_{wl} : la raideur est donc bien une propriété géométrique (forme du plan de flottaison) et non une masse ajoutée.

L'inertie totale en tangage est :

$$I_{\text{tot}} = I_{\text{nav}} + I_{55} = 68\,333 + 121\,688 = 190\,021 \text{ kg}\cdot\text{m}^2 \quad (34)$$

La fréquence propre et la période propre de tangage sont :

$$\omega_\theta = \sqrt{\frac{C_{55}}{I_{\text{tot}}}} = \sqrt{\frac{1\,345\,147}{190\,021}} = 2.66 \text{ rad/s} \quad (35)$$

$$T_\theta = \frac{2\pi}{\omega_\theta} = \frac{2\pi}{2.66} = 2.36 \text{ s} \quad (36)$$

Une période de tangage de 2.36 s correspond à une fréquence de 0.42 Hz.

2.3.4 Cas test : planing hull

Le cas test du *planing hull* est étudié afin de valider les résultats de flottabilité et de géométrie obtenus numériquement. En effet la coque étudiée est celle du tutoriel *planingHull* d'*OpenFOAM v10*. Les principales dimensions du modèle sont :

$$\text{Longueur} = 1.9 \text{ m}, \quad \text{Largeur} = 0.428 \text{ m}, \quad \text{Hauteur} = 0.2 \text{ m}$$

On peut obtenir le volume total sur *Rhino3D* :

$$V = 0.11 \text{ m}^3$$

La masse associée du solide sur *OpenFOAM* est :

$$m = 16.2 \text{ kg}$$

avec $\rho_w = 1025 \text{ kg/m}^3$ et $g = 9.81 \text{ m/s}^2$.

La masse maximale flottable est alors :

$$m_{\text{max}} \approx 110 \text{ kg}$$

Le centre de gravité (centroïde du volume) est situé en (0.8, 0, 0.12).

Le volume immergé est également estimé : $V_i = \frac{m}{\rho_w} \approx 0.015 \text{ m}^3$.

Le tirant d'eau est d'environ : $T \approx 0.027 \text{ m}$.

3 Modélisation numérique

Dans cette Section, nous étudions numériquement la stabilité de plusieurs structures. Dans un premier temps, celle du cas test en comparant les résultats obtenus sur *OpenFOAM* et *Flow3D*, puis la stabilité de deux structures : une boîte rectangulaire et une coque planante.

Dans toute cette partie, on placera la structure flottante dans une masse d'eau de mer, puis à l'aide de conditions initiales (vague, courant etc.) nous en analyserons la stabilité. L'écoulement

étudié sera incompressible et turbulent.

3.1 Cas test cuboïde

Reprenons le cas étudié précédemment en Section 2.3.1, celui d'un cube (Figure 4).

L'objectif de cette partie est de comparer les logiciels *OpenFOAM* et *Flow3D* afin de valider les résultats obtenus avec ce dernier et de vérifier s'il fournit des résultats fiables, à l'instar d'*OpenFOAM*, reconnu pour ses performances en CFD.

3.1.1 Etude OpenFOAM

Afin d'obtenir des résultats sur la stabilité de notre cube flottant, nous avons décidé de nous baser sur le tutoriel *floatingObject* présent dans *OpenFOAM v13*.

Tout d'abord, nous allons expliciter le modèle et les équations utilisées par le logiciel pour obtenir des résultats.

Modélisation RANS et introduction du modèle $k-\varepsilon$:

Une approche classique pour décrire les écoulements turbulents consiste à s'intéresser aux grandeurs moyennes de l'écoulement plutôt qu'aux fluctuations instantanées. Cette approche conduit aux équations de Reynolds moyennées, dites RANS (Reynolds-Averaged Navier–Stokes). Elle repose sur une décomposition des variables physiques en une composante moyenne et une fluctuation turbulente. Ainsi, la vitesse et la pression s'écrivent :

$$U = \bar{U} + u', \quad P = \bar{P} + p' \quad (37)$$

En introduisant cette décomposition dans les équations de Navier–Stokes et en appliquant une moyenne temporelle, on obtient les équations gouvernant l'écoulement moyen d'un fluide incompressible : équations de quantité de mouvement et de conservation de la masse :

$$\frac{\partial(\rho\bar{U})}{\partial t} + \nabla \cdot (\rho\bar{U}\bar{U}) = -\nabla\bar{P} + \mu\nabla^2\bar{U} - \nabla \cdot (\rho\overline{u'u'}) + \rho g \quad (38)$$

$$\nabla \cdot \bar{U} = 0 \quad (39)$$

Le terme supplémentaire $-\rho\overline{u'u'}$, appelé tenseur des contraintes de Reynolds, traduit l'influence des fluctuations turbulentes sur l'écoulement moyen. Il introduit une inconnue supplémentaire dans le système, rendant nécessaire l'utilisation d'un modèle de fermeture.

Dans certaines situations où les statistiques de l'écoulement varient dans le temps, on parle d'approche URANS (Unsteady RANS), qui conserve cette même formulation tout en autorisant

une dépendance temporelle des grandeurs moyennes.

Afin de fermer le système d'équations, une modélisation des contraintes de Reynolds est requise. l'hypothèse de Boussinesq est couramment utilisée : elle établit une analogie avec le comportement d'un fluide newtonien en introduisant une viscosité turbulente μ_t . Le tenseur de Reynolds s'écrit alors :

$$\tau_R = \mu_t \left(\nabla \bar{U} + (\nabla \bar{U})^T \right) - \frac{2}{3} \rho k I \quad (40)$$

où k représente l'énergie cinétique turbulente définie par :

$$k = \frac{1}{2} u' \cdot u' \quad (41)$$

La résolution de l'écoulement moyen repose alors sur la détermination de k et de la viscosité turbulente μ_t . Plusieurs modèles de turbulence existent pour assurer cette fermeture. Parmi eux, les modèles à deux équations offrent un bon compromis entre précision et coût de calcul.

Le modèle $k - \varepsilon$, introduit par Launder et Spalding, repose sur la résolution de deux équations de transport : l'une pour l'énergie cinétique turbulente k , et l'autre pour son taux de dissipation ε .

$$\frac{\partial k}{\partial t} + \mathbf{U} \cdot \nabla k = \nabla \cdot \left[\left(\nu + \frac{\nu_t}{\sigma_k} \right) \nabla k \right] + P_k - \varepsilon \quad (42)$$

$$\frac{\partial \varepsilon}{\partial t} + \mathbf{U} \cdot \nabla \varepsilon = \nabla \cdot \left[\left(\nu + \frac{\nu_t}{\sigma_\varepsilon} \right) \nabla \varepsilon \right] + C_{1\varepsilon} \frac{\varepsilon}{k} P_k - C_{2\varepsilon} \frac{\varepsilon^2}{k} \quad (43)$$

où P_k représente le terme de production de l'énergie cinétique turbulente, et ν_t la viscosité turbulente définie par :

$$\nu_t = C_\mu \rho \frac{k^2}{\varepsilon} \quad (44)$$

où C_μ est une constante empirique généralement fixée à 0.09. Ce modèle est particulièrement adapté aux écoulements à nombre de Reynolds élevé et reste largement utilisé en ingénierie malgré certaines limitations, notamment dans les zones de forts gradients de pression ou de séparation d'écoulement.

Dans *OpenFOAM*, les constantes utilisées par défaut sont :

$$C_\mu = 0.09, C_{1\varepsilon} = 1.44, C_{2\varepsilon} = 1.92, \sigma_k = 1.0, \sigma_\varepsilon = 1.3$$

Le logiciel utilise ces constantes afin de résoudre les équations (42) et (43) de la manière suivante :

Dans le fichier **fvSchemes** on discrétise chacun des termes des équations selon différents schémas numériques adaptés :

- Les dérivées temporelles avec un algorithme d'Euler
- Les gradients avec un algorithme linéaire
- Les divergences (termes de convection) avec des algorithmes de : vanLeer pour la vitesse (U), interfaceCompression vanLeer 1 pour la fraction volumique (α), upwind pour la turbulence (k et ε) et linéaire pour les termes visqueux
- Les laplaciens (termes de diffusion) avec un algorithme linéaire corrigé
- Les interpolations réalisées par le logiciel sont de type linéaires
- On impose un gradient normal aux faces

Ensuite, dans le fichier **fvSolution** on résout les équations discrétisées avec l'algorithme PIMPLE permettant de résoudre des problèmes stationnaires et instationnaires, multiphasiques avec de la turbulence, ainsi que des maillages complexes.

Conditions aux bords :

Avant de lancer la simulation, on définit les conditions entourant notre objet. On se place dans une boîte de 1 m de côtés dont la face du haut modélise l'atmosphère, c'est-à-dire qu'on y impose la pression atmosphérique. Les 5 autres faces sont considérées comme des murs fixes. L'objet flottant quant à lui sera en mouvement : on choisit d'autoriser seulement 2 degrés de liberté, la translation selon l'axe vertical vers le haut (Pz) ainsi que la rotation en roulis (Ry).

De plus, on place initialement une masse d'eau (en rouge sur la Figure 10) au fond de la boîte d'une hauteur de 0.5368 m, puis on en ajoute une seconde à 0.6 m de hauteur (selon z) de 0.8 m de largeur (selon y).

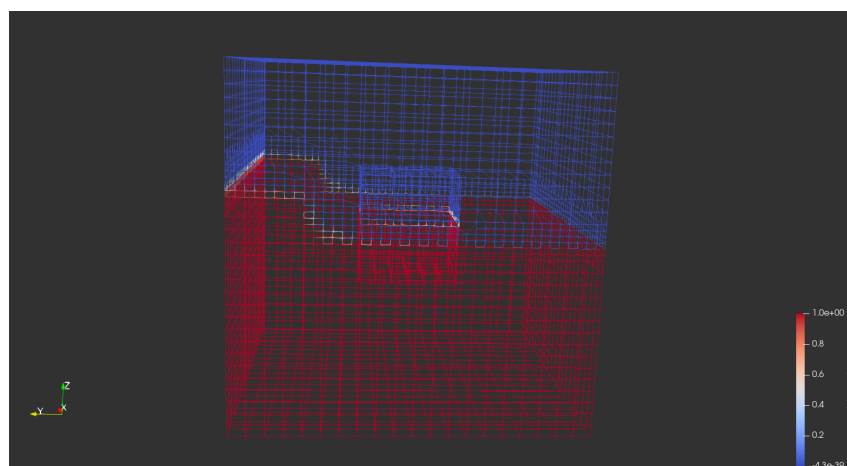


FIGURE 10 : Visualisation des conditions initiales sur *ParaView*

Propriétés physiques :

On se place dans un domaine biphasique air-eau avec :

eau : viscosité cinématique constante $\nu_w = 1 \cdot 10^{-6}$ m²/s, $\rho_w = 1025$ kg/m³.

air : viscosité cinématique constante $\nu_a = 1.48 \cdot 10^{-5}$ m²/s, $\rho_a = 1$ kg/m³.

Pour le cube, on garde les dimensions de la Section 2.3.1 et on impose son centre de masse en (0, 0, 0), puis on le translate en (0.5, 0.45, 0.1) avec la commande :

transform (1 0 0 0 1 0 0 0 1) (0.5 0.45 0.1);

Paramètres de simulation :

Comme décrit précédemment, *OpenFOAM* utilise le modèle de turbulence $k - \varepsilon$ ainsi que la gravité pour la simulation numérique de la stabilité de notre cube.

De plus, on impose un temps de simulation de 6 secondes avec un pas de temps de 0.01 secondes qui s'ajuste tout au long de la simulation afin de ne pas faire exploser la condition de Courant-Friedrich-Levy (CFL) que l'on fixe au maximum à 1.

Géométrie du domaine et maillage :

Il est important de bien choisir la géométrie du problème ainsi que le maillage, car c'est celui-ci qui donnera la précision des résultats ainsi que le temps de calcul.

Etant donné que notre objet flottant fait $0.3 \times 0.3 \times 0.3$ m de côtés, nous choisissons un domaine maillé de $1 \times 1 \times 1$ m avec 12 000 cellules (Figure 11), ce qui n'est pas grand mais permet d'obtenir des résultats corrects tout en optimisant le temps de calcul.

De plus, le cube est considéré comme un solide rigide avec un maillage dynamique autour de lui. On impose également un facteur de relaxation de 0.7 pour l'accélération.

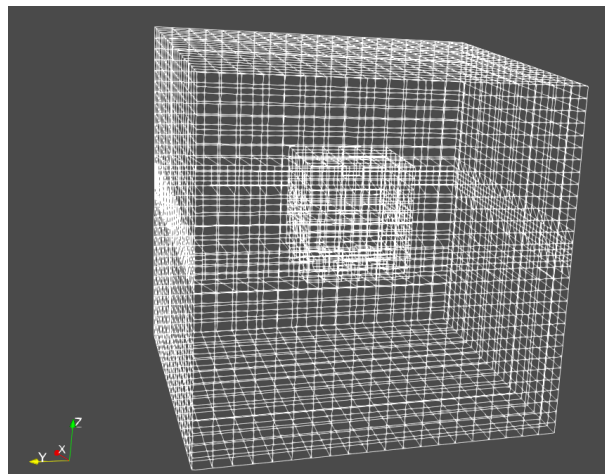


FIGURE 11 : Visualisation du maillage sur *ParaView*

PostProcessing – Résultats :

A l'aide du logiciel *ParaView*, il est possible de visualiser les résultats obtenus une fois la simulation *OpenFOAM* lancée.

Un manière plus précise de visualiser les résultats est de récupérer les fichiers .dat générés dans le dossier **PostProcessing**. Grâce à celui-ci, on récupère : position angulaire, moments, forces visqueuses et de pression, vitesse angulaire et linéaire ainsi que la position du centre de rotation de l'objet flottant.

Ces résultats obtenus sur *OpenFOAM* nous seront utiles pour les comparer à ceux obtenus sur *Flow3D Hydro*.

A l'aide d'un programme écrit en *Python* (Annexe A) nous pouvons bien observer les degrés de libertés bloqués et imposé dans le logiciel. En effet, le cube flottant a tendance à tourner autour de l'axe y (Figure 12) et à translater selon l'axe z (Figure 13), ce qui est donc bien cohérent. En outre, le centre de rotation du cube se situe bien en $(0.5, 0.45, 0.1)$ à l'instant initial.

Ainsi, la rotation présente des oscillations qui diminuent progressivement, traduisant un amortissement dû aux effets visqueux et turbulents. Concernant la position verticale du centre de rotation, cette dernière présente une phase transitoire suivie d'une stabilisation. Ce comportement démontre un équilibre progressif entre le poids, la poussée d'Archimède et la portance hydrodynamique.

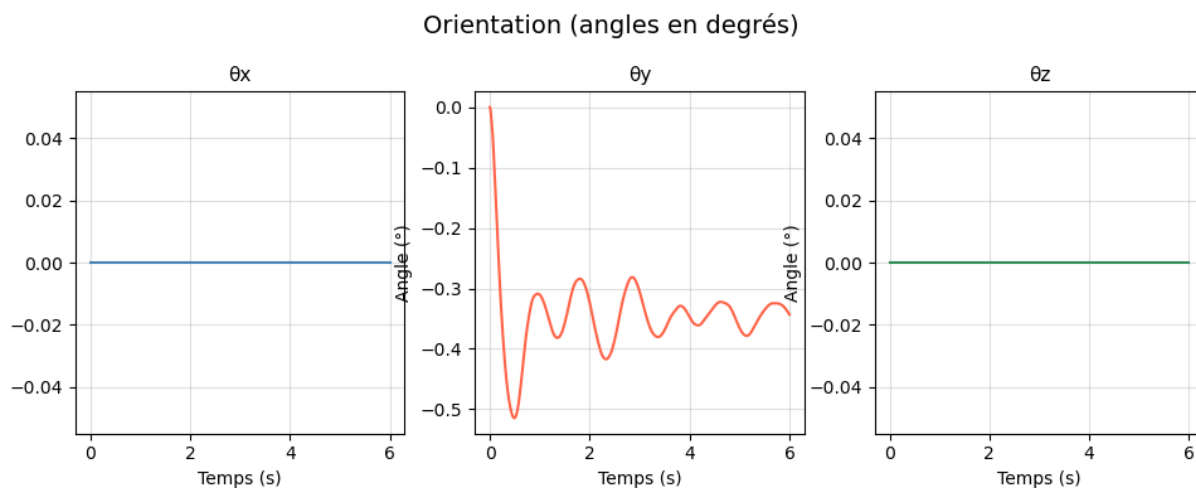


FIGURE 12 : Position angulaire

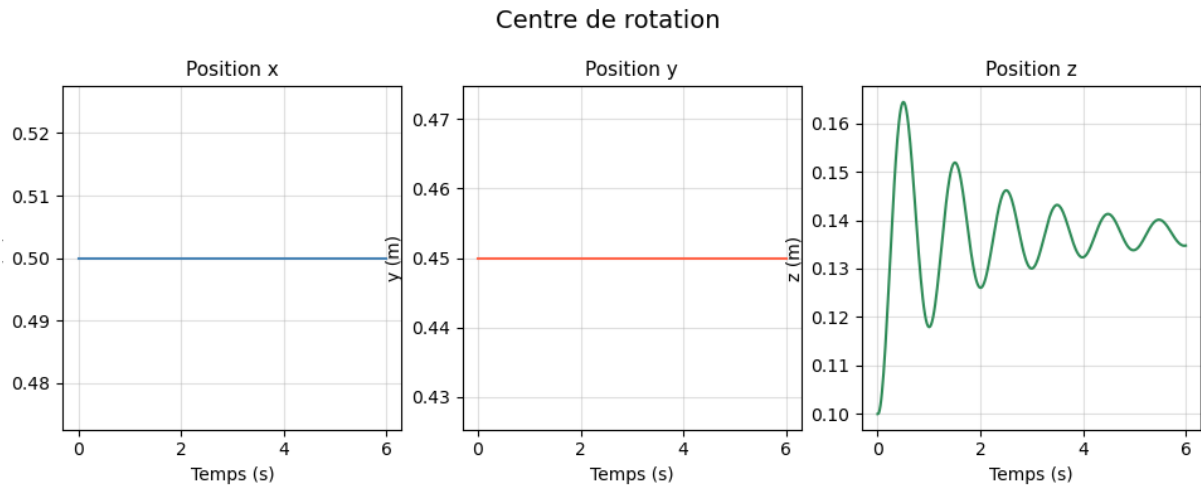


FIGURE 13 : Position du centre de rotation

L'analyse des forces verticales (dominantes ici) (Figure 14) montre une forte instabilité initiale, liée aux effets inertiels et à la masse ajoutée. Globalement, on distingue trois phases : une phase initiale fortement transitoire, une phase oscillatoire amortie, puis un régime stable. Ce comportement confirme la bonne prise en compte des interactions fluide-structure dans la simulation.

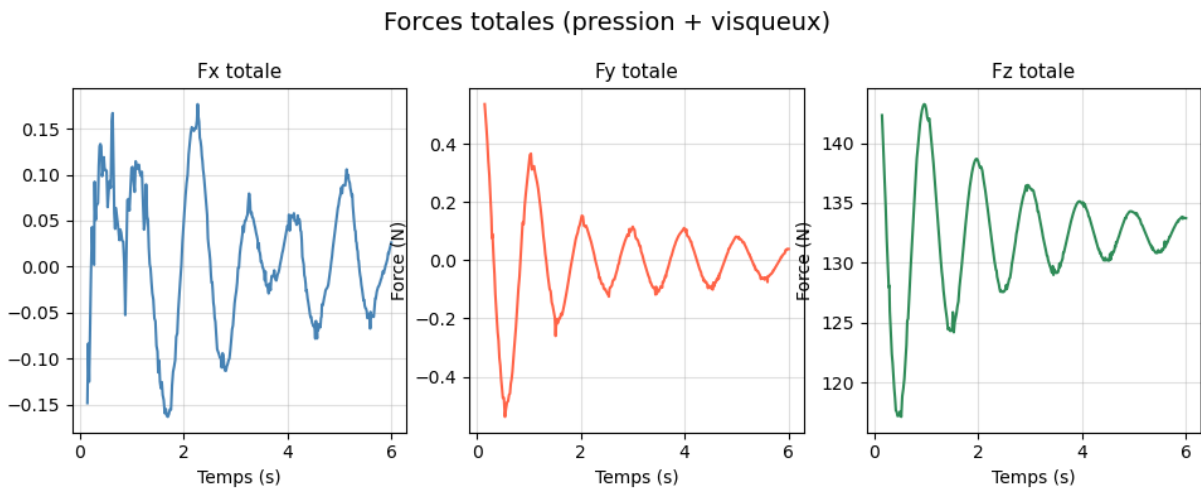


FIGURE 14 : Forces totales

3.1.2 Etude Flow3D

Dans le cadre de cette étude, le logiciel *Flow3D Hydro* est utilisé afin de simuler l'écoulement autour d'une coque planante. *Flow3D* résout les équations de Navier-Stokes en utilisant une méthode des volumes finis sur un maillage cartésien, avec un couplage entre géométrie, surface libre et dynamique du fluide.

Méthode FAVOR : La géométrie est représentée via la méthode FAVOR, qui introduit des fractions de volume V_F et de surface A_x, A_y, A_z dans chaque cellule.

D'un point de vue numérique, cette approche permet d'intégrer les obstacles directement dans les équations discrétisées sans modifier le maillage (Figure 15). Les flux sont pondérés par ces fractions, ce qui garantit la conservation des grandeurs physiques.

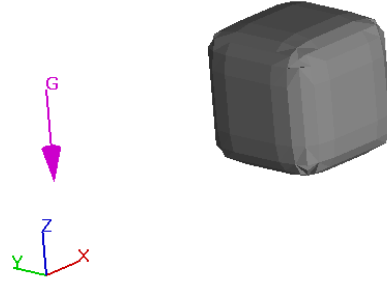


FIGURE 15 : Génération du cube approximé avec FAVOR

Méthode VOF : La surface libre est suivie par la méthode VOF, basée sur une équation de transport :

$$\frac{\partial F}{\partial t} + \nabla \cdot (Fu) = 0$$

Où F représente la fraction volumique de fluide et u sa vitesse.

Le schéma utilisé est de type Donor-Acceptor, qui est un schéma amont (upwind) conservatif.

Équation de continuité :

$$V_F \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho u A) = R_{DIF} + R_{SOR}$$

Avec :

- V_F : le volume de fluide dans la cellule
- A : la surface de la cellule
- R_{DIF} : le terme diffusif
- R_{SOR} : le terme source

Pour un fluide incompressible :

$$\nabla \cdot (uA) = \frac{R_{SOR}}{\rho}$$

où ρ est supposé constant.

Cette équation est discrétisée par la méthode des volumes finis, garantissant la conservation locale de la masse.

Équations de quantité de mouvement :

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \mathbf{G} + \mathbf{f}$$

où $\mathbf{u}$ est la vitesse du fluide, t le temps, p la pression, ρ la masse volumique, $\mathbf{G}$ les forces volumiques (ex : gravité) et $\mathbf{f}$ les forces visqueuses.

Les termes convectifs sont discrétisés avec un schéma upwind, stable pour les écoulements convectifs dominants. Les termes diffusifs utilisent une discrétisation centrée.

Champ de pression et correction : La pression est obtenue via une équation de Poisson dérivée de la continuité :

$$S(p, \mathbf{u}) = 0$$

où p est la pression et $\mathbf{u}$ la vitesse.

Elle est résolue par une méthode itérative de type SOR (Successive Over-Relaxation) :

$$p^{n+1} = p^n + \omega \delta p$$

où p^n est la pression à l'itération n , p^{n+1} celle à l'itération suivante, ω le facteur de relaxation, et δp la correction de pression.

Cette approche s'inscrit dans une méthode de projection, où :

- une vitesse intermédiaire est calculée,
- la pression corrige ensuite cette vitesse pour satisfaire la continuité.

Mise à jour des vitesses :

$$u^{n+1} = u^n + \frac{\delta p \Delta t}{\rho \Delta x} \quad ; \quad v^{n+1} = v^n + \frac{\delta p \Delta t}{\rho \Delta y} \quad ; \quad w^{n+1} = w^n + \frac{\delta p \Delta t}{\rho \Delta z}$$

où u, v, w sont les composantes de la vitesse selon x, y, z , Δt le pas de temps, ρ la masse volumique, et $\Delta x, \Delta y, \Delta z$ les dimensions des cellules du maillage.

Le schéma temporel est explicite, ce qui impose une condition de stabilité de type CFL.

Modélisation de la turbulence : L'écoulement est modélisé en régime turbulent via le modèle RNG $k - \varepsilon$. Les équations sont moyennées (RANS), et la turbulence est prise en compte via une viscosité turbulente :

$$\mu_t = \rho C_\mu \frac{k^2}{\varepsilon}$$

où μ_t est la viscosité turbulente, ρ la masse volumique, C_μ une constante empirique, k l'énergie cinétique turbulente et ε son taux de dissipation.

Ce modèle est adapté aux écoulements avec forts gradients de vitesse et zones de séparation, typiques des coques planantes. La surface libre est advectée à chaque pas de temps avec le champ de vitesse mis à jour.

Couplage fluide-structure : Dans le cas d'un objet mobile, les forces hydrodynamiques sont calculées puis injectées dans les équations du solide :

$$M \frac{dV}{dt} = F \quad ; \quad J \frac{d\omega}{dt} = T$$

où M est la masse du solide, $\mathbf{V}$ sa vitesse, $\mathbf{F}$ la force hydrodynamique résultante, $\mathbf{J}$ le tenseur d'inertie, ω la vitesse angulaire et $\mathbf{T}$ le moment (torque).

Le couplage est explicite et repose sur une mise à jour séquentielle fluide - solide.

Nous allons à présent simuler la stabilité de notre cube dans les mêmes conditions que sous *OpenFOAM*, afin de comparer les résultats obtenus avec deux logiciels différents.

Paramétrage du logiciel : *Flow3D* comporte différents onglets à remplir afin d'obtenir une simulation CFD correcte, dont les fonctions sont :

- **Global :** Modification de la durée de simulation. Ici, nous choisissons une durée de 6 secondes.
- **Physics :** Choix des modèles physiques à activer. Dans notre cas, nous choisissons gravité, objets en mouvements ainsi que turbulence et viscosité. Pour la turbulence, nous choisissons le modèle RAS ($k - \varepsilon$).
- **Fluids :** Déterminer les propriétés du fluide présent lors de la simulation, soit ici de l'eau salée à 20°C de densité $\rho_w = 1025 \text{ kg/m}^3$.
- **Geometry :** Importation du fichier .stl de notre cube afin de définir la géométrie. Nous définissons également les propriétés de mouvement de cet objet : pas de translation selon x et y , pas de rotation selon x et z et une densité $\rho = 500 \text{ kg/m}^3$. Cette partie permet également de déplacer l'objet et de redéfinir sa taille si le fichier .stl ne correspond pas aux grandeurs du logiciel.
- **Mesh :** Définition du maillage et du domaine entourant le cube. Ici, comme sur *OpenFOAM* nous choisissons un domaine de 1 mètre de côtés, centré autour de notre cube. De plus, nous rentrons également une taille de cellules de maillage correspondant au nombre de cellules sur *OpenFOAM* soit environ 12 000 (Figure 16).

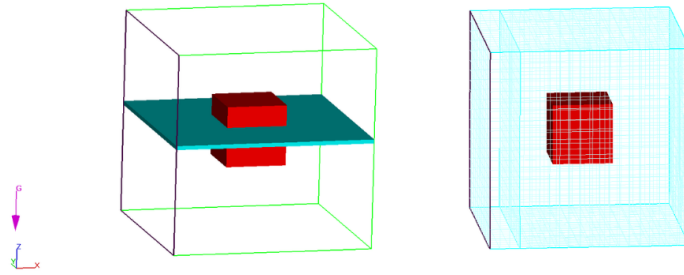


FIGURE 16 : Géométrie et maillage du domaine

- **Boundaries :** Détermination des conditions aux limites pour le maillage défini auparavant. Chaque face possède une condition spécifique (Figure 17). Nous avons décidé de créer 2 maillages afin de reproduire au mieux la condition d'ajout d'une masse d'eau supplémentaire sur le côté. En effet, cela semble être la meilleure manière de représenter le cas étudié sous *OpenFOAM*. Dans ce souci de modélisation, les conditions aux limites deviennent :

- x_{min} : Wall
- x_{max} : Wall
- y_{min} : Wall
- y_{max} : Wall
- z_{min} : Wall
- z_{max} : Pressure

puis

- x_{min} : Pressure
- x_{max} : Wall
- y_{min} : Wall
- y_{max} : Wall
- z_{min} : Wall
- z_{max} : Pressure

En x_{min} du deuxième domaine, on impose une hauteur d'eau de 0.6 m afin de créer un mouvement de l'eau.

En z_{max} on impose la pression de l'air afin de simuler l'interface air-eau.

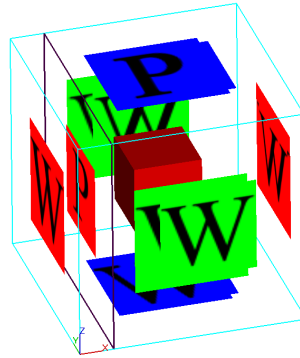


FIGURE 17 : Visualisation des conditions aux bords

- **Initial** : Choix des conditions initiales : l'eau au repos est à 0.538 m de hauteur.
- **Output** : Détermination des grandeurs de sortie à obtenir : les forces, le centre de flottabilité ainsi que la hauteur métacentrique. En effet, le logiciel permet automatiquement d'obtenir les autres grandeurs de base (déplacement, vitesse, vitesse angulaire, etc.).

Une fois la simulation lancée, on obtient les premiers résultats à visualiser sur *Flow3D Post*. Puis, en extrayant les données en fichier .csv et à l'aide d'un programme *Python* nous pouvons obtenir les courbes des grandeurs pertinentes.

Visualisation sur *Flow3D-Post* :

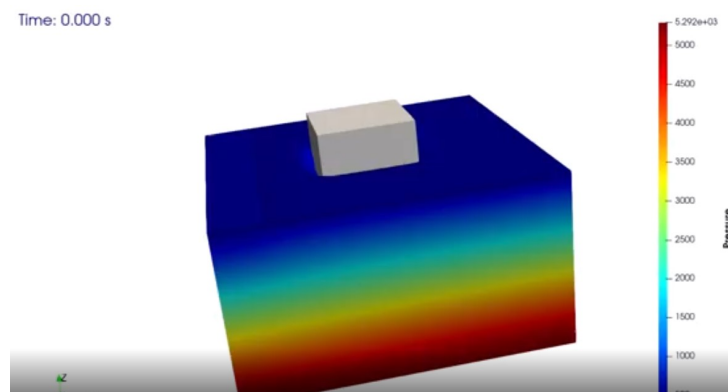


FIGURE 18 : Simulation FLOW-3D (cliquer pour voir la vidéo)

Première comparaison avec la théorie : Une première grandeur à comparer est la hauteur métacentrique (GM) que nous n'avons pas pu obtenir avec *OpenFOAM* mais que nous avons calculé en Section 2.3.2.

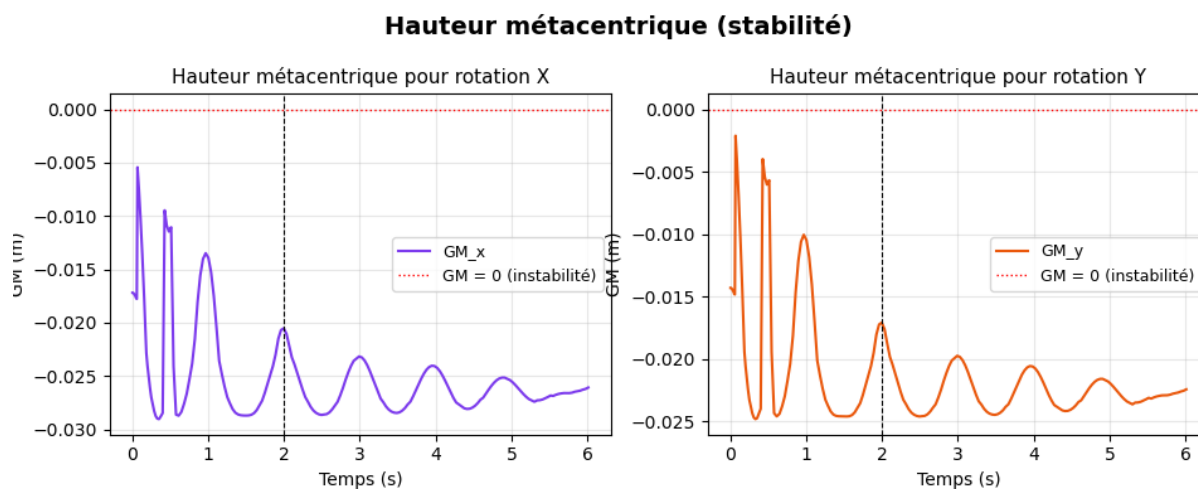


FIGURE 19 : Hauteur métacentrique obtenue sous Flow3D

La Figure 19 permet de voir que tout au long de la simulation le GM oscille jusqu'à se stabiliser autour de -0.025 m lorsque le cube revient à sa position d'équilibre. Ceci est bien cohérent avec l'étude théorique réalisée en Section 2.3.1, ce qui veut dire que *Flow3D* traduit bien la stabilité de notre cube au repos.

3.1.3 Comparaison des logiciels

Comparons à présent plus précisément les résultats obtenus sur chacun des logiciels de CFD.

La Figure 20 montre le déplacement vertical du centre de gravité. Ici, on observe une bonne cohérence entre les courbes qui se superposent : elles oscillent à une fréquence de $1 H_z$ bien qu'elles ne soient pas de même amplitude. En effet, les conditions initiales choisies sur *Flow3D* ne sont pas identiquement les mêmes que sur *OpenFOAM*, ce qui semble fausser l'amplitude de translation de cube.

Les résultats décrits précédemment semblent s'appliquer de la même manière pour la vitesse verticale ainsi que les forces verticales (Figures 21 et 22).

Finalement, nous pouvons dire que les résultats obtenus avec les deux logiciels sont cohérents. Cela prouve que *Flow3D Hydro* fournit des résultats "réalistes" et cohérents avec la théorie, ce qui ne décrédibilise pas son utilisation en tant qu'alternative à *OpenFOAM*. En effet, la prise en main de ce logiciel de CFD est relativement rapide et facile, d'autant que le temps de simulation en est raccourci.

En revanche, il convient d'augmenter significativement le nombre de mailles sur *Flow3D* afin d'obtenir des résultats de meilleure qualité car on observe des instabilités numériques dans les résultats.

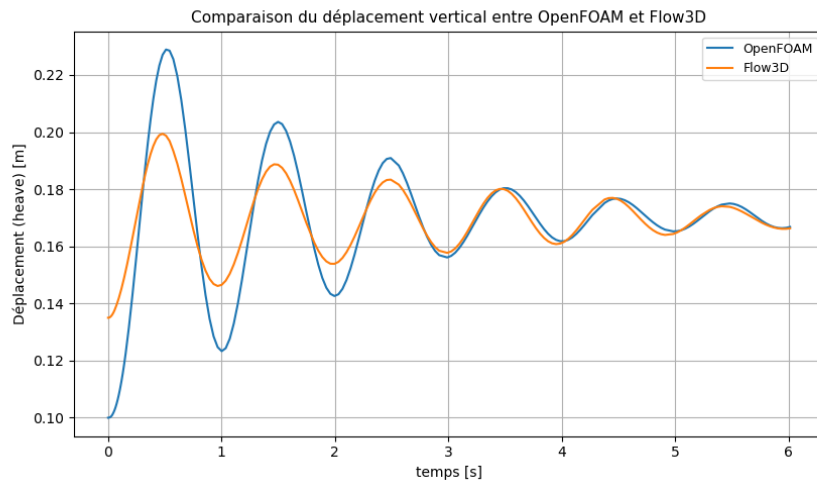


FIGURE 20 : Comparaison du déplacement vertical

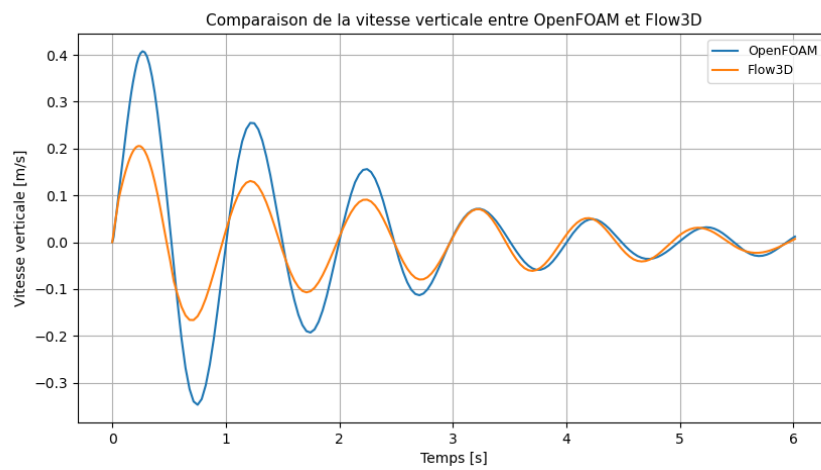


FIGURE 21 : Comparaison de la vitesse verticale

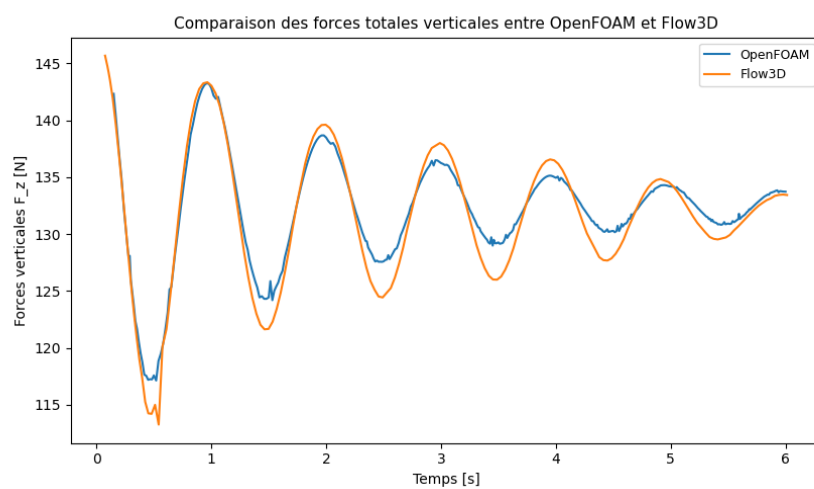


FIGURE 22 : Comparaison des forces verticales

3.2 Cas test : planing hull

Reprenons à présent notre deuxième cas test détaillé en Section 2.3.4 permettant également de valider *Flow3D*.

3.2.1 Paramètres des simulations

De même que précédemment, nous effectuons une simulation sous *OpenFOAM* et *Flow3D* en utilisant les mêmes paramètres pour les deux logiciels, détaillés ci-après.

Conditions initiales et paramètres de simulation :

On impose en entrée une vitesse uniforme du fluide différente de $\mathbf{U} = (-4.598, 0, 0)$.

- Modèle de mouvement du solide :
 - Masse : 16.146 kg
 - Inertie : (0.4, 0, 0, 5, 0, 5)
 - Degrés de liberté : translation verticale (Pz), rotation en tangage (Ry) comme les modèles précédents.
- Solveur utilisé : **interFoam** (écoulement diphasique incompressible avec surface libre)
- Temps de simulation : de 0 à 5 secondes
- Pas de temps initial : $\Delta t = 10^{-4}$ s
- Ajustement automatique du pas de temps : activé (**adjustTimeStep yes**)
- Nombre de Courant maximal : **maxCo** = 5
- Nombre de Courant pour l'interface : **maxAlphaCo** = 5

Géométrie du domaine et maillage :

On définit un domaine rectangulaire tridimensionnel dont les dimensions sont :

- $x \in [-14, 6]$
- $y \in [-5, 0]$
- $z \in [-7, 3]$
- Maillage structuré de type **blockMesh** composé de 6 blocs superposés verticalement
- Raffinement du maillage selon la direction verticale afin de mieux résoudre la surface libre
- Nombre de cellules : 11 680

- direction x : 20
- direction y : 4
- direction z : 146 cellules distribuées non uniformément (raffinement proche de la surface libre)
- Étirement du maillage (`simpleGrading`) :
 - raffinement près de l'interface air/eau
 - maillage plus grossier loin de la zone d'intérêt

Conditions aux limites du domaine :

- `inlet` : condition de type `patch` (entrée du fluide)
- `outlet` : condition de type `patch` (sortie du fluide)
- `atmosphere` : condition de type `patch` (surface libre supérieure)
- `bottom` : condition de symétrie (`symmetryPlane`)
- `side` : condition de symétrie (`symmetryPlane`)
- `midPlane` : condition de symétrie (`symmetryPlane`)

3.2.2 Résultats et comparaison

Après simulation sur les deux logiciels de CFD, nous pouvons observer la coque planante qui avance à une vitesse constante de 4.6 m/s.

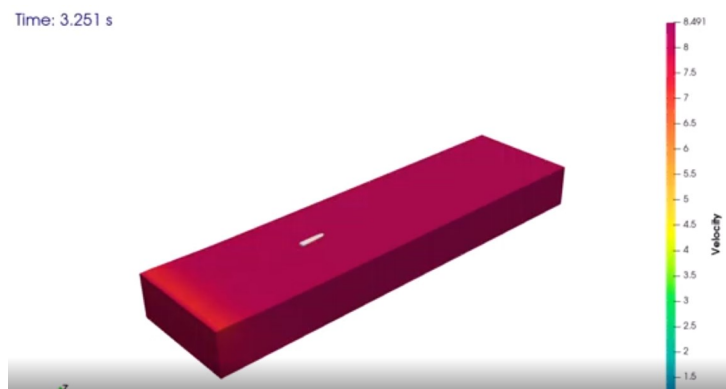


FIGURE 23 : Simulation FLOW-3D (cliquer pour voir la vidéo)

Remarque : La vidéo présentée ne correspond pas à la simulation à une vitesse de 4.6 m/s, mais à 8.2 m/s.

Après récupération des données, il est pertinent de comparer les forces verticales pour chacune des simulations. En effet, nous pouvons voir que la force de portance est de 150 N pour *OpenFOAM* et de 320 N pour *Flow3D* (Figure 24). Les forces sont du même ordre de grandeur et

traduisent une bonne cohérence entre les résultats des deux logiciels bien que *Flow3D* introduise des petites instabilités de la coque planante tout au long de la simulation.

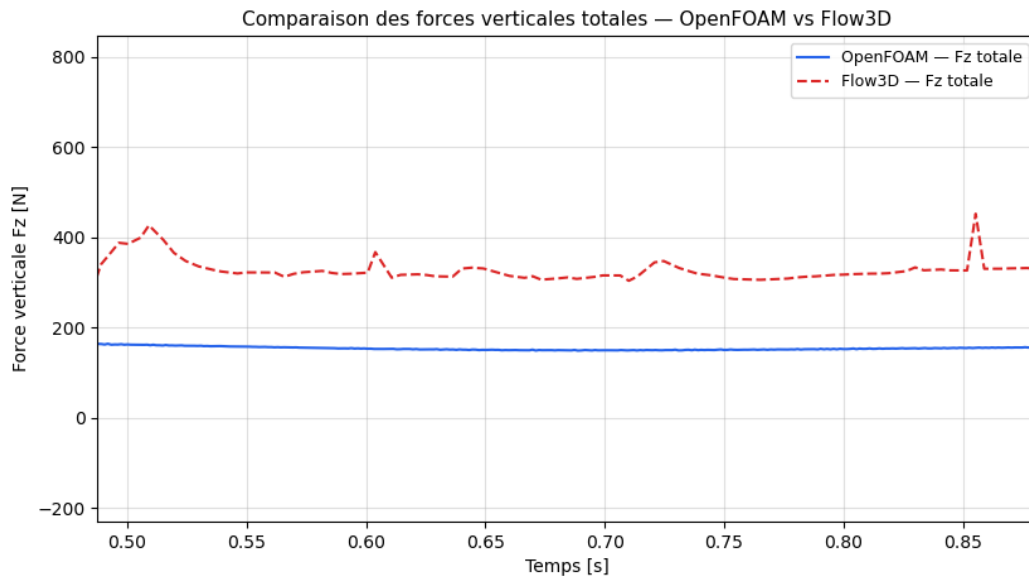


FIGURE 24 : Comparaison des forces verticales

3.3 Parallélépipède rectangle

Afin de comparer les résultats obtenus avec la coque planante, une géométrie simplifiée de type parallélépipède rectangle est étudiée. Les paramètres physiques sont identiques à ceux utilisés précédemment (Section 2.3.2), à l'exception du maillage qui est adapté à cette nouvelle configuration. On gardera les mêmes conditions pour la coque planante hormis le maillage.

Conditions de simulation :

- Fluide : *Use fluid volume*
- Vitesse : de 0 à 20 m/s (augmentation de 1 m/s par seconde)
- Pas de temps : 1 s
- Durée totale : 20 s

Domaine de simulation : Le domaine de calcul est défini de manière à limiter l'influence des conditions aux limites sur l'écoulement autour du solide :

- $x \in [-55; 55]$
- $y \in [-7.5; 7.5]$
- $z \in [-21; 11]$

Le volume total d'eau est d'environ $34\,650\text{ m}^3$.

Le domaine est dimensionné de sorte à reproduire un environnement quasi-infini :

- 5 fois la longueur en amont (entrée),
- 2 fois la largeur sur les côtés,
- 1 fois la longueur au-dessus (élévation possible du bateau),
- 2 fois la longueur en dessous.

Maillage : Un raffinement du maillage est appliqué au voisinage de la géométrie afin d'améliorer la précision des calculs :

- Taille des cellules (extérieur) : 1 m
- Taille des cellules (intérieur) : 0.5 m
- Nombre total de mailles : 69 300

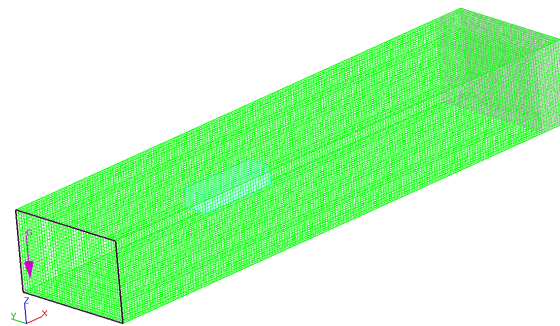


FIGURE 25 : Visualisation du maillage sur *Flow3D*

Conditions aux limites : Les conditions aux limites sont définies comme suit :

- Entrée : vitesse imposée
- Sortie : *wave outflow* (absorption des ondes)
- Surface supérieure : pression atmosphérique
- Côtés et fond : conditions de symétrie

Les conditions de symétrie permettent de simuler un écoulement en domaine ouvert, assimilable à une grande étendue d'eau (type océan), tout en réduisant le coût de calcul. Le dimensionnement du domaine garantit que les effets de bord restent négligeables sur la dynamique du fluide autour de la structure.

3.4 Coque planante

On s'intéresse à la simulation d'un écoulement incompressible autour de notre coque planante dessinée sur *Rhino3D*, ses dimensions sont donc celles explicitées en Section 2.3.3.

3.4.1 Etude de la convergence

Afin d'étudier la convergence du maillage, on réalise une simulation courte de 2s avec la géométrie dessinée sur *Rhino3d*. On choisit de comparer la vitesse angulaire à l'instant final pour 5 tailles de maille différentes. Nous traçons l'évolution de l'erreur E en fonction du nombre de cellules N dans un diagramme en échelle logarithmique. L'erreur est définie comme la différence absolue entre la solution numérique et la solution de référence obtenue pour la plus fine discrétisation ($h = 0.25$).

On suppose une loi de convergence de la forme : $E \sim N^{-p}$

ce qui conduit, après passage au logarithme : $\log(E) = -p \log(N) + C$

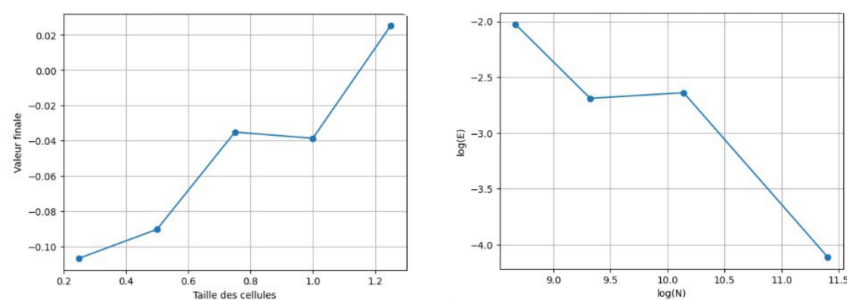


FIGURE 26 : Erreur en fonction de la taille des cellules, $\log(E)$ en fonction de $\log(\text{nombre de cellules})$

À partir des données numériques, une estimation de la pente donne : $p \approx 0.76$

On en déduit que le schéma présente un ordre de convergence d'environ 0.76 (Figure 33). Cette valeur est légèrement inférieure à 1 : ordre de convergence attendu pour un logiciel de simulation. *Flow3D* remplit donc les attentes, bien que la convergence observée ne soit pas parfaite.

3.4.2 Flow 3D

De même que précédemment, nous effectuons une simulation sous *OpenFOAM* et *Flow3D* en utilisant les mêmes paramètres pour les deux logiciels, détaillés ci-après.

Conditions de simulation :

- Fluide : *Use fluid volume*
- Vitesse : de 0 à 20 m/s (augmentation de 1 m/s par seconde)

- Pas de temps : 1 s
- Durée totale : 20 s

Domaine de simulation : Le domaine de calcul est défini de manière à limiter l'influence des conditions aux limites sur l'écoulement autour du solide :

- $x \in [-55; 55]$
- $y \in [-7.5; 7.5]$
- $z \in [-21; 11]$

Le volume total d'eau est d'environ 34 650 m³.

Le domaine est dimensionné de sorte à reproduire un environnement quasi-infini :

- 5 fois la longueur en amont (entrée),
- 2 fois la largeur sur les côtés,
- 1 fois la longueur au-dessus (élévation possible du bateau),
- 2 fois la longueur en dessous.

Maillage : Un raffinement du maillage est appliqué au voisinage de la géométrie afin d'améliorer la précision des calculs :

- Taille des cellules (extérieur) : 0.7 m
- Taille des cellules (intérieur) : 0.4 m
- Nombre total de mailles : 380 000

Conditions aux limites : Les conditions aux limites sont définies comme suit :

- Entrée : vitesse imposée
- Sortie : *wave outflow* (absorption des ondes)
- Surface supérieure : pression atmosphérique
- Côtés et fond : conditions de symétrie

Equation et modèle : On utilise dorénavant la turbulence RNG. Les équations de quantité de mouvement s'écrivent alors sous la forme :

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nabla \cdot [(\mu + \mu_t) \nabla \mathbf{u}] + \mathbf{G}$$

avec la viscosité turbulente définie par le modèle RNG :

$$\mu_t = \rho C_\mu \frac{k^2}{\varepsilon}$$

où k représente l'énergie cinétique turbulente et ε son taux de dissipation.

Résultats : La simulation permet d'observer un régime transitoire où le bateau avance et commence à se lever, puis une phase de marsouinage à mesure que la vitesse augmente.

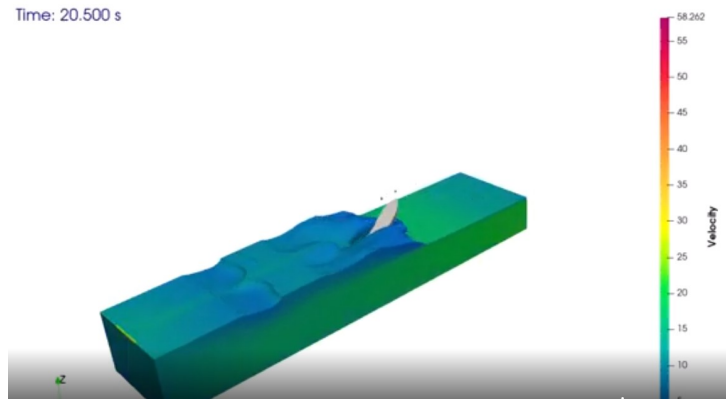


FIGURE 27 : Simulation FLOW-3D (cliquer pour voir la vidéo)

Analyse des coefficients hydrodynamiques :

Les résultats obtenus montrent une diminution progressive du coefficient de portance C_L avec l'augmentation de la vitesse, ainsi qu'une relative stabilité du coefficient de traînée C_D en régime établi. Ce comportement est caractéristique des coques planantes : à mesure que la vitesse augmente, la portance hydrodynamique permet une réduction de la surface mouillée, ce qui entraîne une diminution de C_L . Parallèlement, la traînée, bien que croissante en valeur absolue, reste modérée en coefficient du fait de la normalisation par la pression dynamique.

Ces observations sont en accord avec les résultats issus d'études CFD récentes, notamment celles publiées dans le *Journal of Marine Science and Engineering*, qui mettent en évidence une diminution du coefficient de portance avec le nombre de Froude et une stabilisation du coefficient de traînée en régime planant. Elles sont également cohérentes avec les travaux classiques de Savitsky [12], qui décrivent la réduction de la surface mouillée et l'évolution de la portance dans le régime de planing.

3.5 Comparaison et discussion des résultats

Comparons à présent les résultats obtenus pour deux géométries différentes : le parallélépipède rectangle ainsi que la coque planante dessinée sur *Rhino3D*.

Géométrie rectangulaire :

- Coefficient de portance moyen : $\overline{C_L} = 0.0235$
- Coefficient de traînée moyen : $\overline{C_D} = 0.0123$
- Rapport portance/traînée : $\overline{L/D} = 3.43$

Coque *Rhino3D* :

- Coefficient de portance moyen : $\overline{C_L} = 0.0525$
- Coefficient de traînée moyen : $\overline{C_D} = 0.0122$
- Rapport portance/traînée : $\overline{L/D} = 4.77$

Analyse comparative :

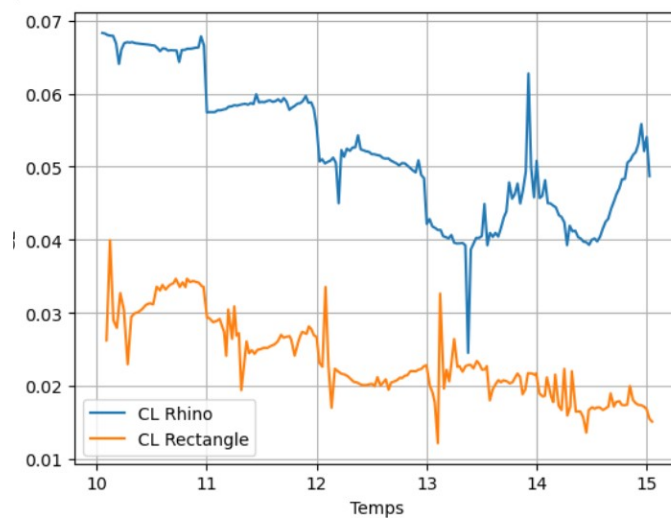


FIGURE 28 : Coefficient de portance en fonction du temps

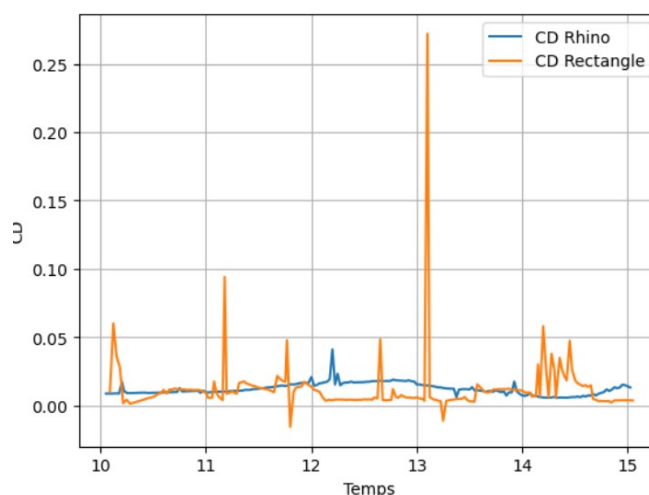
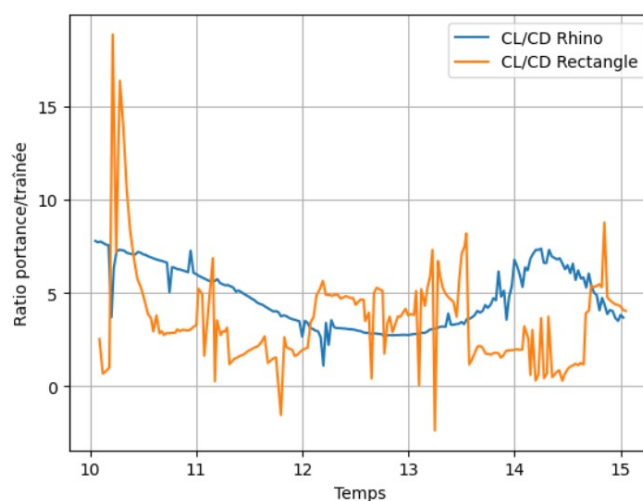


FIGURE 29 : Coefficient de traînée en fonction du temps

FIGURE 30 : Ratio L/D en fonction du temps

La coque Rhino présente une portance moyenne environ deux fois supérieure à celle de la géométrie rectangulaire, pour une traînée similaire. Cela se traduit par une amélioration significative du rendement hydrodynamique, avec un rapport L/D supérieur d'environ 40%.

Ces résultats mettent en évidence l'influence de la géométrie sur les performances hydrodynamiques, la forme profilée de la coque Rhino favorisant un écoulement plus efficace et une meilleure génération de portance.

L'analyse des vitesses angulaires montre que la coque Rhino présente une dynamique plus intense, avec une vitesse angulaire moyenne environ deux fois supérieure à celle de la géométrie rectangulaire et une énergie cinématique près de trois fois plus élevée. Cela traduit une réponse hydrodynamique plus marquée aux sollicitations du fluide.

La géométrie rectangulaire présente des mouvements plus faibles mais également moins structurés, ce qui peut être associé à une performance hydrodynamique globale inférieure, en cohérence avec les résultats de portance et de rendement L/D .

Discussion des résultats :

Les coefficients moyens de portance et de traînée ont été calculés pour deux configurations : une coque de type Rhino et une géométrie rectangulaire. L'étude de Matveev [8] montre que l'introduction d'un vrillage de la coque permet d'améliorer significativement les performances hydrodynamiques. En particulier, la portance est augmentée tandis que la surface mouillée est réduite, ce qui conduit à une amélioration du rapport portance/traînée. Ces résultats sont en accord avec ceux obtenus dans notre étude, où la coque Rhino présente une efficacité supérieure à la géométrie rectangulaire.

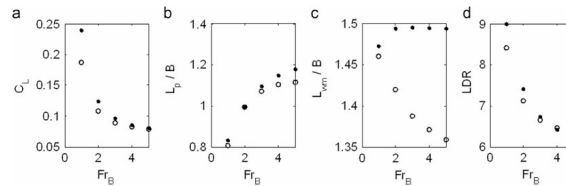


Fig. 5. (a) Lift coefficient, (b) center of pressure, (c) mean wetted length, and (d) lift-drag ratio. Filled circles, twisted hull; open circles, "equivalent" monohedral hull.

FIGURE 31 : Matveev [8] Coefficient de portance en fonction du nombre de Froude

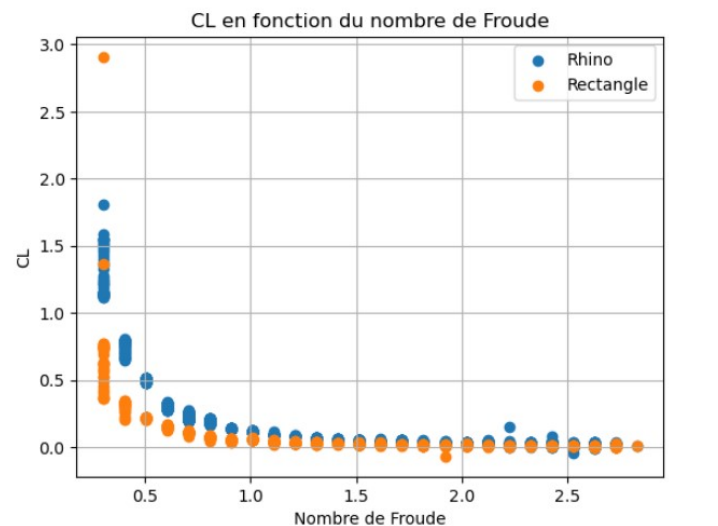


FIGURE 32 : Coefficient de portance en fonction du nombre de Froude de nos coques

On obtient la même forme de courbe qui converge vers un coefficient de portance très faible.

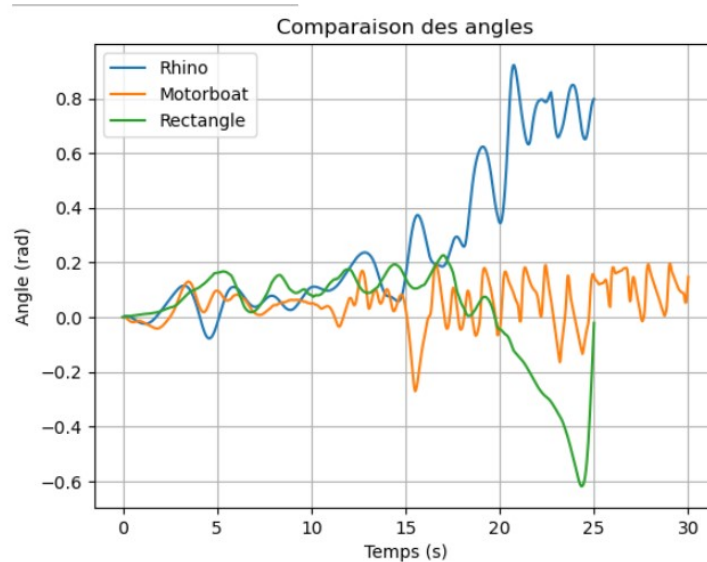


FIGURE 33 : Angle en degré en fonction du temps pour 3 géométries différentes

L'analyse temporelle des angles met en évidence des comportements dynamiques très différents selon la géométrie. La coque Rhino présente une augmentation progressive de l'angle, traduisant une génération de portance efficace mais associée à une dynamique marquée. À l'inverse, la géométrie rectangulaire montre une instabilité importante avec une divergence de l'angle, indiquant un comportement hydrodynamique non maîtrisé. Enfin, le cas motorboat présente des oscillations de faible amplitude et de haute fréquence autour d'une position d'équilibre, caractéristique d'un système stable mais moins performant.

4 Conclusion

En conclusion, cette étude nous a permis de mieux comprendre le phénomène de marsouinage sur les coques planantes, en combinant une approche théorique et une modélisation numérique avancée.

Dans un premier temps, une approche théorique a permis de caractériser la stabilité hydrostatique et hydrodynamique de différentes géométries, mettant en évidence le rôle central de paramètres tels que la position du centre de gravité, la surface de flottaison et la répartition des volumes. Ces analyses ont montré que la stabilité dépend fortement de la géométrie de la coque, notamment sur les performances hydrodynamiques, et des conditions extérieures.

Dans un second temps, des simulations numériques ont été réalisées à l'aide des logiciels *OpenFOAM* et *Flow3D*. La comparaison des résultats a montré une bonne cohérence globale entre les deux outils, validant ainsi la pertinence des modèles utilisés. *Flow3D* s'est révélé plus accessible en termes de prise en main, tandis qu'*OpenFOAM* offre une plus grande flexibilité de paramétrage. Avec *Flow3D*, il est toutefois nécessaire d'utiliser un maillage plus fin que sous *OpenFOAM* afin d'éviter les divergences numériques et de garantir la convergence de la solution.

L'étude a également mis en évidence l'importance des effets dynamiques, notamment la masse ajoutée et les interactions fluide-structure, dans le comportement en tangage. Le phénomène de marsouinage apparaît comme une instabilité complexe résultant d'un couplage entre portance hydrodynamique et mouvement du navire, fortement influencé par la vitesse et la configuration géométrique.

La géométrie rectangulaire présente une masse ajoutée plus élevée que la coque Rhino, ce qui conduit à un comportement équivalent à celui d'un système plus lourd, donc plus inertiel. La coque Rhino montre des réponses fréquentielles plus stables, tandis que le rectangle présente des oscillations moins structurées. Enfin, les coefficients hydrodynamiques confirment la meilleure efficacité de la coque Rhino, avec un CL plus élevé et un meilleur rapport L/D, alors que la géométrie rectangulaire génère davantage de traînée pour une portance plus faible.

Enfin, ce travail souligne l'intérêt de combiner approches théoriques et simulations numériques pour mieux comprendre et prédire le comportement des coques planantes. Des perspectives d'amélioration pourraient inclure une validation expérimentale, une étude paramétrique plus poussée ou l'analyse de configurations encore plus réalistes, afin d'optimiser la conception de navires rapides et stables.

Références

- [1] Donald BLOUNT et Louis CODEGA. “Dynamic stability of planing boats”. In : *Marine Technology* 29 (1992), p. 4-12.
- [2] Etienne DUFOUR. *Naval Architecture and Ship Hydrodynamics*. Notes de cours, SeaTech Université de Toulon. 2025.
- [3] Ahmed O. ELAGHBASH. “A CFD Study of the Resistance Behavior of a Planing Hull in Restricted Waterways”. In : *Sustainable Marine Structures* 03 (2021), p. 32-53.
- [4] S. HIRDARIS et al. “A review on the hydrodynamics of planing hulls”. In : *Ocean Engineering* 303 (2024), p. 1-38.
- [5] H. ISLAM et al. “OpenFOAM analysis of the wave radiation by a box-type floating structure”. In : *Ocean Engineering* 193 (2019), p. 1-13.
- [6] J. H. JUNG et al. “Numerical simulation of wave interacting with a free rolling body”. In : *Naval Architecture and Ocean Engineering* (2013), p. 333-347.
- [7] D. J. KIMA, K. P. RHEEB et Y. J. YOUB. “Theoretical prediction of running attitude of a semi-displacement round bilge vessel at high speed”. In : *Applied Ocean Research* 41 (2013), p. 41-47.
- [8] Konstantin I. MATVEEV. “CFD Study on Hydrodynamic Performances of a Planing Hull”. In : *Journal of Marine Science and Engineering* (2022).
- [9] A. NAJAFI et al. “The hydrodynamic resistance of stepped planing hulls under different geometrical and physical conditions”. In : *Scientific Journals of the Maritime University of Szczecin* (2019), p. 24-31.
- [10] Dominique PAULET, Dominique PRESLES et Frédéric NEUMAN. “Architecture Navale”. In : *Connaissance et Pratique* (2018).
- [11] Marcos RODRÍGUEZ et Johannes SPINNEKEN. “A laboratory study on the loading and motion of a heaving box”. In : *Journal of Fluids and Structures* (2016), p. 107-126.
- [12] Daniel SAVITSKY. “Hydrodynamic Design of Planing Hulls”. In : *Marine Technology* 1 (1964), p. 71-95.
- [13] Delft University of TECHNOLOGY. “OFFSHORE HYDROMECHANICS”. In : *First Edition January* (2001).
- [14] Det Norske VERITAS. “Environmental conditions and environmental loads”. In : *DNV-RP-C205* (Décembre 2019), p. 243-244.
- [15] Y. ZHANG et al. “Study on the porpoising performance of high-speed vessels : Experimental, 3D numerical, and 2.5D numerical approach”. In : *Ocean Engineering* 341 (2025), p. 1-15.

A Annexe

Les programmes écrits en Python ci-après permettent d'analyser les données obtenues sur *OpenFOAM* et *Flow3D Hydro* après simulation.

Listing 1 : Post-processing des données *OpenFOAM* et *Flow3D Hydro* pour vitesse et déplacement du cube

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import re
4 import pandas as pd
5 import matplotlib.gridspec as gridspec
6 from scipy.fft import fft, fftfreq
7 from scipy.signal import correlate, correlation_lags
8 from scipy.interpolate import interp1d
9 import warnings
10 warnings.filterwarnings('ignore')
11
12 filename = "C:\\Users\\lisa\\Documents\\SeaTech\\2A-MOCA\\PROJET\\postProcessing_bon\\
    rigidBodyState\\0\\floatingObject.dat"
13
14 def parse_openfoam_motion(filename):
15
16     temps = []
17     centre_rotation = [] # Centre of rotation (x, y, z)
18     orientation = [] # Angles (theta_x, theta_y, theta_z) en degres
19     vitesse_lin = [] # Vitesse lineaire (Vx, Vy, Vz)
20     vitesse_ang = [] # Vitesse angulaire (omega_x, omega_y, omega_z) en deg/s
21
22     vec_re = re.compile(r'\(([^\)]+)\)')
23
24     with open(filename, 'r') as f:
25         lignes = f.readlines()
26
27     for ligne in lignes:
28         ligne = ligne.strip()
29         if not ligne or ligne.startswith('#'):
30             continue
31
32         try:
33             t = float(ligne.split()[0])
34         except ValueError:
35             continue
36
37         vecs = vec_re.findall(ligne)
38         # vecs[0] -> centre of rotation (x y z)
39         # vecs[1] -> orientation (theta_x theta_y theta_z) [deg]
40         # vecs[2] -> vitesse lineaire (Vx Vy Vz)
41         # vecs[3] -> vitesse angulaire (omega_x omega_y omega_z) [deg/s]
42
43         if len(vecs) < 4:
44             print(f"Ligne ignoree ({len(vecs)} vecteurs) : {ligne[:60]}...")
45             continue
46
47         try:
48             def to_vec(s):
49                 return [float(x) for x in s.split()]
50

```

```

51         cr = to_vec(vecs[0])
52         ori = to_vec(vecs[1])
53         vl = to_vec(vecs[2])
54         va = to_vec(vecs[3])
55
56         if not all(len(v) == 3 for v in [cr, ori, vl, va]):
57             print(f"Vecteur mal forme à t={t}")
58             continue
59
60         temps.append(t)
61         centre_rotation.append(cr)
62         orientation.append(ori)
63         vitesse_lin.append(vl)
64         vitesse_ang.append(va)
65
66         except (ValueError, IndexError) as e:
67             print(f"Erreur de parsing à t={t} : {e}")
68             continue
69
70     data = {
71         'time' : np.array(temps),
72         'centre_rotation' : np.array(centre_rotation),
73         'orientation' : np.array(orientation), # deg
74         'vitesse_lin' : np.array(vitesse_lin),
75         'vitesse_ang' : np.array(vitesse_ang), # deg/s
76     }
77
78     print(f"\n{len(temps)} pas de temps lus avec succès.")
79     print(f" Plage temporelle : [{data['time'][0]:.4f}, {data['time'][-1]:.4f}] s")
80     return data
81
82 # Lecture
83
84 print("Lecture du fichier...")
85 data = parse_openfoam_motion(filename)
86
87 if len(data['time']) == 0:
88     raise RuntimeError("Aucune donnée lue vérifiez le chemin du fichier.")
89
90 t1 = data['time']
91 CR = data['centre_rotation']
92 ORI = data['orientation']
93 VL = data['vitesse_lin']
94 VA = data['vitesse_ang']
95
96 labels_xyz = ['x', 'y', 'z']
97 colors = ['steelblue', 'tomato', 'seagreen']
98
99 # Figure : Trajectoire du centre de rotation
100 fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharex=True)
101 fig.suptitle('Centre de rotation', fontsize=14)
102
103 for i, (lbl, col) in enumerate(zip(labels_xyz, colors)):
104     axes[i].plot(t1, CR[:, i], color=col, linewidth=1.5)
105     axes[i].set_title(f'Position {lbl}')
106     axes[i].set_xlabel('Temps (s)')
107     axes[i].set_ylabel(f'{lbl} (m)')
108     axes[i].grid(True, alpha=0.4)
109
110 plt.tight_layout()
111 plt.savefig('centre_rotation.png', dpi=150)

```

```

112 plt.show()
113
114 FILE_EXCEL= "C:\\Users\\lisa\\Documents\\SeaTech\\2A-MOCA\\PROJET\\
        tab_floating_object_RAS.xlsx"
115
116 print("=" * 60)
117 print("LECTURE DU FICHIER EXCEL")
118 print("=" * 60)
119
120 df_raw = pd.read_excel(FILE_EXCEL, header=0)
121
122 col_map = {
123     'Time' : 't',
124     'Floating Structure: Angular X-velocity; Body System' : 'omega_x',
125     'Floating Structure: Angular Y-velocity; Body System' : 'omega_y',
126     'Floating Structure: Angular Z-velocity; Body System' : 'omega_z',
127     'Floating Structure: CM X-displacement' : 'disp_x',
128     'Floating Structure: CM Y-displacement' : 'disp_y',
129     'Floating Structure: CM Z-displacement' : 'disp_z',
130     'Floating Structure: Metacentric Height For X-rotation' : 'meta_x',
131     'Floating Structure: Metacentric Height For Y-rotation' : 'meta_y',
132     'Floating Structure: X-pressure Force; Space System' : 'Fp_x',
133     'Floating Structure: X-shear Force; Space System' : 'Fv_x',
134     'Floating Structure: Y-pressure Force; Space System' : 'Fp_y',
135     'Floating Structure: Y-shear Force; Space System' : 'Fv_y',
136     'Floating Structure: Z-pressure Force; Space System' : 'Fp_z',
137     'Floating Structure: Z-shear Force; Space System' : 'Fv_z',
138 }
139 df = df_raw.rename(columns=col_map)
140 df['t'] = pd.to_numeric(df['t'], errors='coerce')
141 df = df.dropna(subset=['t']).reset_index(drop=True)
142
143 # Forces totales
144 df['Ft_x'] = df['Fp_x'] + df['Fv_x']
145 df['Ft_y'] = df['Fp_y'] + df['Fv_y']
146 df['Ft_z'] = df['Fp_z'] + df['Fv_z']
147
148 t = df['t'].values
149 print(f" {len(df)} pas de temps lus [{t[0]:.4f}, {t[-1]:.4f}] s")
150 print(f" Colonnes disponibles : {list(df.columns)}")
151
152 # Masque regime etabli
153 T_START_REGIME=2
154 mask = t >= T_START_REGIME
155 if mask.sum() < 10:
156     print(f" Trop peu de points apres t={T_START_REGIME}s utilisation totale.")
157     mask = np.ones(len(t), dtype=bool)
158 df_r = df[mask].copy()
159 t_r = df_r['t'].values
160 print(f" Régime établi : {mask.sum()} points (t {T_START_REGIME} s)")
161
162 plt.rcParams.update({'font.size': 10, 'axes.titlesize': 11,
163                     'axes.labelsize': 10, 'legend.fontsize': 9})
164
165 C = {'blue': '#2563EB', 'red': '#DC2626', 'green': '#16A34A',
166      'orange': '#EA580C', 'purple': '#7C3AED', 'gray': '#6B7280'}
167
168 # Figure : Vue d'ensemble temporelle
169 fig = plt.figure(figsize=(16, 12))
170 fig.suptitle('Vue d\'ensemble temporelle Objet flottant (RAS)',
171             fontsize=14, fontweight='bold')

```

```

172 gs = gridspec.GridSpec(3, 2, figure=fig, hspace=0.5, wspace=0.35)
173
174 donnees_ts = [
175     ('Ft_z', 'Force verticale Fz (N)', C['blue'], 'Fp_z', 'Fz pression'),
176     ('Ft_y', 'Force latérale Fy (N)', C['red'], None, None),
177     ('disp_z', 'Déplacement z Heave (m)', C['green'], None, None),
178     ('omega_y', 'Vitesse angulaire y (rad/s)', C['orange'], None, None),
179     ('meta_x', 'Hauteur métacentrique GM_x (m)', C['purple'], None, None),
180     ('meta_y', 'Hauteur métacentrique GM_y (m)', C['gray'], None, None),
181 ]
182 positions = [(0,0),(0,1),(1,0),(1,1),(2,0),(2,1)]
183
184 for (col, titre, couleur, col2, lbl2), (r, c) in zip(donnees_ts, positions):
185     ax = fig.add_subplot(gs[r, c])
186     ax.plot(t, df[col].values, color=couleur, lw=1.3, label=titre.split('(')[0].strip())
187     if col2:
188         ax.plot(t, df[col2].values, color=C['green'], lw=0.8, alpha=0.6, label=lbl2)
189     ax.axvline(T_START_REGIME, color='k', ls='--', lw=0.8,
190               label=f't_régime={T_START_REGIME}s')
191     ax.set_title(titre); ax.set_xlabel('Temps (s)')
192     ax.legend(fontsize=8); ax.grid(True, alpha=0.3)
193
194 plt.savefig('fig1_vue_ensemble.png', dpi=150, bbox_inches='tight')
195 plt.show()
196
197 from scipy.interpolate import interp1d
198
199 f_interp = interp1d(t1, CR[:, 2], bounds_error=False, fill_value="extrapolate")
200
201 CR_interp = f_interp(t)
202 tab=[0.1035 for i in range(len(df['disp_z']))]
203 df['disp_z']=df['disp_z']+tab
204
205 plt.figure()
206 plt.plot(t, CR_interp, label="OpenFOAM")
207 plt.plot(t, df['disp_z'].values, label="Flow3D")
208
209 plt.xlabel("temps [s]")
210 plt.ylabel("Déplacement (heave) [m]")
211 plt.title("Comparaison du déplacement vertical entre OpenFOAM et Flow3D")
212 plt.legend()
213 plt.grid()
214 plt.show()
215
216 plt.figure()
217 plt.plot(t, df['Ft_z'])
218 plt.show()
219
220 # Figure : Orientation (angles)
221 fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharex=True)
222 fig.suptitle('Orientation (angles en degrés)', fontsize=14)
223
224 for i, (lbl, col) in enumerate(zip(labels_xyz, colors)):
225     axes[i].plot(t1, ORI[:, i], color=col, linewidth=1.5)
226     axes[i].set_title(f'{lbl}')
227     axes[i].set_xlabel('Temps (s)')
228     axes[i].set_ylabel('Angle (°)')
229     axes[i].grid(True, alpha=0.4)
230
231 plt.tight_layout()
232 plt.savefig('orientation.png', dpi=150)

```

```

233 plt.show()
234
235 # Figure : Vitesse lineaire
236 fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharex=True)
237 fig.suptitle('Vitesse linéaire', fontsize=14)
238
239 for i, (lbl, col) in enumerate(zip(labels_xyz, colors)):
240     axes[i].plot(t1, VL[:, i], color=col, linewidth=1.5)
241     axes[i].set_title(f'V{lbl}')
242     axes[i].set_xlabel('Temps (s)')
243     axes[i].set_ylabel('Vitesse (m/s)')
244     axes[i].grid(True, alpha=0.4)
245
246 plt.tight_layout()
247 plt.savefig('vitesse_lineaire.png', dpi=150)
248 plt.show()
249
250 # ===== #
251
252 FILL = "C:\\Users\\lisa\\Documents\\SeaTech\\2A-MOCA\\PROJET\\vitesse_floating_object_RAS
253 .xlsx"
254
255 df = pd.read_excel(FILL)
256
257 df = df.rename(columns={
258     'Time': 'tt',
259     'Floating Structure: CM Z-velocity': 'v_z'
260 })
261
262 df['tt'] = pd.to_numeric(df['tt'], errors='coerce')
263 df = df.dropna(subset=['tt']).reset_index(drop=True)
264
265 tt = df['tt'].values
266
267 plt.figure(figsize=(10, 5))
268
269 # ===== #
270
271 from scipy.interpolate import interp1d
272 import matplotlib.pyplot as plt
273
274 # interpolation de la vitesse verticale simulation
275 f_vz = interp1d(t1, VL[:, 2],
276                 bounds_error=False,
277                 fill_value="extrapolate")
278
279 VLz_interp = f_vz(tt)
280
281 plt.figure(figsize=(10, 5))
282
283 plt.plot(tt, VLz_interp, label='OpenFOAM')
284 plt.plot(tt, df['v_z'], label='Flow3D')
285
286 plt.xlabel("Temps [s]")
287 plt.ylabel("Vitesse verticale [m/s]")
288 plt.title("Comparaison de la vitesse verticale entre OpenFOAM et Flow3D")
289 plt.legend()
290 plt.grid()
291
292 plt.show()
293
294 # Figure : Vitesse angulaire

```

```

293 fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharex=True)
294 fig.suptitle('Vitesse angulaire', fontsize=14)
295
296 for i, (lbl, col) in enumerate(zip(labels_xyz, colors)):
297     axes[i].plot(t1, VA[:, i], color=col, linewidth=1.5)
298     axes[i].set_title(f'{lbl}')
299     axes[i].set_xlabel('Temps (s)')
300     axes[i].set_ylabel('Vitesse angulaire ( $\dot{x}$ /s)')
301     axes[i].grid(True, alpha=0.4)
302
303 plt.tight_layout()
304 plt.savefig('vitesse_angulaire.png', dpi=150)
305 plt.show()
306
307 # Figure 5 : Deplacement vertical et angle de tangage
308 fig, axes = plt.subplots(2, 1, figsize=(10, 8), sharex=True)
309 fig.suptitle('Mouvement principal de l\'objet flottant', fontsize=14)
310
311 axes[0].plot(t1, CR[:, 2], color='steelblue', linewidth=2)
312 axes[0].set_ylabel('Position z (m)')
313 axes[0].set_title('Déplacement vertical (heave)')
314 axes[0].grid(True, alpha=0.4)
315
316 axes[1].plot(t, ORI[:, 1], color='tomato', linewidth=2)
317 axes[1].set_ylabel('Angle thetay ( $\dot{x}$ )')
318 axes[1].set_xlabel('Temps (s)')
319 axes[1].set_title('Angle de tangage (pitch)')
320 axes[1].grid(True, alpha=0.4)
321
322 plt.tight_layout()
323 plt.savefig('heave_pitch.png', dpi=150)
324 plt.show()
325
326 # Statistiques
327 print("\n Deplacement vertical z (heave) ")
328 print(f" z_0 : {CR[0, 2]:.6f} m")
329 print(f" z_max : {np.max(CR[:, 2]):+.6f} m")
330 print(f" z_min : {np.min(CR[:, 2]):+.6f} m")
331 print(f" Amplitude crete-a-crete : {np.max(CR[:, 2]) - np.min(CR[:, 2]):.6f} m")
332
333 print("\n Angle de tangage theta y (pitch)")
334 print(f" thetay_max : {np.max(ORI[:, 1]):+.4f}  $\dot{x}$ ")
335 print(f" thetay_min : {np.min(ORI[:, 1]):+.4f}  $\dot{x}$ ")
336 print(f" Amplitude crete-a-crete : {np.max(ORI[:, 1]) - np.min(ORI[:, 1]):.4f}  $\dot{x}$ ")
337
338 print("\n Vitesse angulaire omegay max")
339 print(f" omegay_max : {np.max(VA[:, 1]):+.4f}  $\dot{x}$ /s")
340 print(f" omegay_min : {np.min(VA[:, 1]):+.4f}  $\dot{x}$ /s")

```

Listing 2 : Post-processing des données *OpenFOAM* et *Flow3D Hydro* pour les forces du cube

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import re
4 import pandas as pd
5 import matplotlib.gridspec as gridspec
6 from scipy.fft import fft, fftfreq
7 from scipy.signal import correlate, correlation_lags
8 from scipy.interpolate import interp1d
9 import warnings
10 warnings.filterwarnings('ignore')

```

```
11
12 filename = "C:\\Users\\lisa\\Documents\\SeaTech\\2A-MOCA\\PROJET\\postProcessing_bon\\
    rigidBodyForces\\0\\rigidBodyForces.dat"
13
14 def parse_openfoam_forces(filename):
15
16     temps = []
17     forces_pressure = []
18     forces_viscous = []
19     moments_pressure = []
20     moments_viscous = []
21
22     # Regex pour extraire 3 flottants à l'intérieur d'une paire de parenthèses
23     vec_re = re.compile(r'\s*([^\s]+)\s*\(')
24
25     with open(filename, 'r') as f:
26         lignes = f.readlines()
27
28     for ligne in lignes:
29         ligne = ligne.strip()
30         if not ligne or ligne.startswith('#'):
31             continue
32
33         try:
34             t = float(ligne.split()[0])
35         except ValueError:
36             continue
37
38         # Trouve tous les vecteurs (groupes de 3 nombres entre parenthèses simples)
39         vecs = vec_re.findall(ligne)
40         # vecs[0] -> CofR
41         # vecs[1] -> forces pressure
42         # vecs[2] -> forces viscous
43         # vecs[3] -> moments pressure
44         # vecs[4] -> moments viscous
45
46         if len(vecs) < 5:
47             print(f"Ligne ignorée (seulement {len(vecs)} vecteurs) : {ligne[:60]}...")
48             continue
49
50         try:
51             def to_vec(s):
52                 return [float(x) for x in s.split()]
53
54             fp = to_vec(vecs[1])
55             fv = to_vec(vecs[2])
56             mp = to_vec(vecs[3])
57             mv = to_vec(vecs[4])
58
59             if not all(len(v) == 3 for v in [fp, fv, mp, mv]):
60                 print(f"Vecteur mal formé sur la ligne t={t}")
61                 continue
62
63             temps.append(t)
64             forces_pressure.append(fp)
65             forces_viscous.append(fv)
66             moments_pressure.append(mp)
67             moments_viscous.append(mv)
68
69         except (ValueError, IndexError) as e:
70             print(f"Erreur de parsing à t={t} : {e}")
```

```

71         continue
72
73     data = {
74         'time' : np.array(temps),
75         'forces_pressure' : np.array(forces_pressure),
76         'forces_viscous' : np.array(forces_viscous),
77         'moments_pressure' : np.array(moments_pressure),
78         'moments_viscous' : np.array(moments_viscous),
79     }
80     data['forces_total'] = data['forces_pressure'] + data['forces_viscous']
81     data['moments_total'] = data['moments_pressure'] + data['moments_viscous']
82
83     print(f"\n{len(temps)} pas de temps lus avec succès.")
84     print(f"   Plage temporelle : [{data['time'][0]:.4f}, {data['time'][-1]:.4f}] s")
85     return data
86
87 print("Lecture du fichier...")
88 data = parse_openfoam_forces(filename)
89
90 if len(data['time']) == 0:
91     raise RuntimeError("Aucune donnée lue vérifiez le chemin du fichier.")
92
93 t2 = data['time']
94 FP = data['forces_pressure']
95 FV = data['forces_viscous']
96 FT = data['forces_total']
97 MP = data['moments_pressure']
98 MV = data['moments_viscous']
99 MT = data['moments_total']
100
101 labels_xyz = ['x', 'y', 'z']
102 colors = ['steelblue', 'tomato', 'seagreen']
103
104 # Figure : Forces totales
105 fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharex=True)
106 fig.suptitle('Forces totales (pression + visqueux)', fontsize=14)
107
108 for i, (lbl, col) in enumerate(zip(labels_xyz, colors)):
109     axes[i].plot(t2, FT[:, i], color=col, linewidth=1.5)
110     axes[i].set_title(f'F{lbl} totale')
111     axes[i].set_xlabel('Temps (s)')
112     axes[i].set_ylabel('Force (N)')
113     axes[i].grid(True, alpha=0.4)
114
115 plt.tight_layout()
116 plt.savefig('forces_totales.png', dpi=150)
117 plt.show()
118
119 FILE_EXCEL = "C:\\Users\\lisa\\Documents\\SeaTech\\2A-MOCA\\PROJET\\
120             tab_floating_object_RAS.xlsx"
121
122 T_START_REGIME = 2.0 # Debut du régime établi (s)
123
124 # Paramètres physiques
125 RHO = 1025.0 # Densité du fluide (kg/$^3$)
126 G = 9.81 # Gravité (m/s$^2$)
127 L_CAR = 0.3 # Longueur caractéristique (m)
128 A_VAGUE = 0.0 # Amplitude de vague (m)
129
130 print(" " * 60)
131 print(" LECTURE DU FICHIER EXCEL")
132 print(" " * 60)

```

```

131
132 df_raw = pd.read_excel(FILE_EXCEL, header=0)
133
134 col_map = {
135     'Time' : 't',
136     'Floating Structure: Angular X-velocity; Body System' : 'omega_x',
137     'Floating Structure: Angular Y-velocity; Body System' : 'omega_y',
138     'Floating Structure: Angular Z-velocity; Body System' : 'omega_z',
139     'Floating Structure: CM X-displacement' : 'disp_x',
140     'Floating Structure: CM Y-displacement' : 'disp_y',
141     'Floating Structure: CM Z-displacement' : 'disp_z',
142     'Floating Structure: Metacentric Height For X-rotation' : 'meta_x',
143     'Floating Structure: Metacentric Height For Y-rotation' : 'meta_y',
144     'Floating Structure: X-pressure Force; Space System' : 'Fp_x',
145     'Floating Structure: X-shear Force; Space System' : 'Fv_x',
146     'Floating Structure: Y-pressure Force; Space System' : 'Fp_y',
147     'Floating Structure: Y-shear Force; Space System' : 'Fv_y',
148     'Floating Structure: Z-pressure Force; Space System' : 'Fp_z',
149     'Floating Structure: Z-shear Force; Space System' : 'Fv_z',
150 }
151 df = df_raw.rename(columns=col_map)
152 df['t'] = pd.to_numeric(df['t'], errors='coerce')
153 df = df.dropna(subset=['t']).reset_index(drop=True)
154
155 # Forces totales
156 df['Ft_x'] = df['Fp_x'] + df['Fv_x']
157 df['Ft_y'] = df['Fp_y'] + df['Fv_y']
158 df['Ft_z'] = df['Fp_z'] + df['Fv_z']
159
160 t = df['t'].values
161 print(f" {len(df)} pas de temps lus [{t[0]:.4f}, {t[-1]:.4f}] s")
162 print(f" Colonnes disponibles : {list(df.columns)}")
163
164 # Masque regime établi
165 mask = t >= T_START_REGIME
166 if mask.sum() < 10:
167     print(f" Trop peu de points après t={T_START_REGIME}s utilisation totale.")
168     mask = np.ones(len(t), dtype=bool)
169 df_r = df[mask].copy()
170 t_r = df_r['t'].values
171 print(f" Régime établi : {mask.sum()} points (t {T_START_REGIME} s)")
172
173
174 def stats(signal, nom, unite):
175     s = {
176         'mean': np.mean(signal), 'std': np.std(signal),
177         'max' : np.max(signal), 'min': np.min(signal),
178         'amp' : np.max(signal) - np.min(signal)
179     }
180     print(f"\n {nom}")
181     print(f" Moyenne : {s['mean']:.4f} {unite}")
182     print(f" Écart-type : {s['std']:.4f} {unite}")
183     print(f" Max / Min : {s['max']:.4f} / {s['min']:.4f} {unite}")
184     print(f" Amplitude : {s['amp']:.4f} {unite}")
185     return s
186
187 def fft_analyse(t_reg, signal, nom):
188     dt = np.mean(np.diff(t_reg))
189     t_u = np.linspace(t_reg[0], t_reg[-1], len(t_reg))
190     sig_u = interp1d(t_reg, signal, kind='linear')(t_u)
191     N = len(sig_u)

```

```

192     freqs = fftfreq(N, d=dt)
193     spec  = np.abs(fft(sig_u - np.mean(sig_u)))
194     pos   = freqs > 0
195     f_p, s_p = freqs[pos], spec[pos]
196     f_dom = f_p[np.argmax(s_p)]
197     T_dom = 1.0 / f_dom if f_dom > 0 else np.inf
198     print(f"      {nom} f dominante : {f_dom:.4f} Hz | T = {T_dom:.4f} s")
199     return f_p, s_p, f_dom, T_dom
200
201 def correlation(t1, s1, t2, s2, n1, n2):
202     t_min = max(t1[0], t2[0]); t_max = min(t1[-1], t2[-1])
203     N      = min(len(t1), len(t2))
204     tc     = np.linspace(t_min, t_max, N)
205     sc1    = interp1d(t1, s1, kind='linear', fill_value='extrapolate')(tc)
206     sc2    = interp1d(t2, s2, kind='linear', fill_value='extrapolate')(tc)
207     sc1n   = (sc1 - sc1.mean()) / (sc1.std() + 1e-12)
208     sc2n   = (sc2 - sc2.mean()) / (sc2.std() + 1e-12)
209     corr   = correlate(sc1n, sc2n, mode='full') / N
210     lags   = correlation_lags(N, N, mode='full') * np.mean(np.diff(tc))
211     idx    = np.argmax(np.abs(corr))
212     dep, cmax = lags[idx], corr[idx]
213     print(f"      {n1} / {n2} Corrélation max : {cmax:+.4f} | Déphasage : {dep:+.4f} s")
214     return lags, corr, dep, cmax
215
216
217 def ratio_pv(fp, fv, nom):
218     ft = np.abs(fp) + np.abs(fv)
219     ft[ft == 0] = np.nan
220     rp = np.nanmean(np.abs(fp) / ft) * 100
221     rv = np.nanmean(np.abs(fv) / ft) * 100
222     print(f"      {nom} Pression : {rp:.1f}% | Cisaillement : {rv:.1f}%")
223     return rp, rv
224
225 print("\n" + "=" * 60)
226 print(f" STATISTIQUES (régime établi t {T_START_REGIME} s)")
227 print("=" * 60)
228
229 print("\n Forces totales")
230 s_Fx = stats(df_r['Ft_x'].values, 'Fx total', 'N')
231 s_Fy = stats(df_r['Ft_y'].values, 'Fy total', 'N')
232 s_Fz = stats(df_r['Ft_z'].values, 'Fz total (portance)', 'N')
233
234 print("\n Mouvement")
235 s_z = stats(df_r['disp_z'].values, 'Déplacement z (heave)', 'm')
236 s_ox = stats(df_r['omega_x'].values, 'Vitesse angulaire x', 'rad/s')
237 s_oy = stats(df_r['omega_y'].values, 'Vitesse angulaire y', 'rad/s')
238
239 print("\n Hauteur métacentrique")
240 s_mx = stats(df_r['meta_x'].values, 'GM pour rotation X', 'm')
241 s_my = stats(df_r['meta_y'].values, 'GM pour rotation Y', 'm')
242
243 print("\n" + "=" * 60)
244 print(" ANALYSE FRÉQUENTIELLE (FFT)")
245 print(" " * 60)
246 f_Fz, s_Fz_fft, fdom_Fz, Tdom_Fz = fft_analyse(t_r, df_r['Ft_z'].values, 'Fz total')
247 f_Fy, s_Fy_fft, fdom_Fy, Tdom_Fy = fft_analyse(t_r, df_r['Ft_y'].values, 'Fy total')
248 f_z, s_z_fft, fdom_z, Tdom_z = fft_analyse(t_r, df_r['disp_z'].values, 'Heave z')
249 f_oy, s_oy_fft, fdom_oy, Tdom_oy = fft_analyse(t_r, df_r['omega_y'].values, 'y')
250
251
252 print("\n" + " " * 60)

```

```

253 print("  CORRÉLATIONS FORCES / MOUVEMENT")
254 print(" " * 60)
255 lags_z, corr_z, dep_z, cmax_z = correlation(
256     t_r, df_r['Ft_z'].values, t_r, df_r['disp_z'].values, 'Fz', 'Heave z')
257 lags_oy, corr_oy, dep_oy, cmax_oy = correlation(
258     t_r, df_r['Ft_y'].values, t_r, df_r['omega_y'].values, 'Fy', 'y')
259
260
261 print("\n" + " " * 60)
262 print("  COEFFICIENTS ADIMENSIONNELS")
263 print(" " * 60)
264 Cz_moy = np.mean(df_r['Ft_z'].values) / (RHO * G * A_VAGUE * L_CAR**2)
265 Cz_amp = s_Fz['amp'] / (RHO * G * A_VAGUE * L_CAR**2)
266 print(f"    Cz moyen      : {Cz_moy:+.4f}")
267 print(f"    Cz amplitude  : {Cz_amp:.4f}")
268
269
270 plt.rcParams.update({'font.size': 10, 'axes.titlesize': 11,
271                     'axes.labelsize': 10, 'legend.fontsize': 9})
272
273 C = {'blue': '#2563EB', 'red': '#DC2626', 'green': '#16A34A',
274     'orange': '#EA580C', 'purple': '#7C3AED', 'gray': '#6B7280'}
275
276 # Figure : Vue d'ensemble temporelle
277 fig = plt.figure(figsize=(16, 12))
278 fig.suptitle('Vue d\'ensemble temporelle Objet flottant (RAS)',
279             fontsize=14, fontweight='bold')
280 gs = gridspec.GridSpec(3, 2, figure=fig, hspace=0.5, wspace=0.35)
281
282 donnees_ts = [
283     ('Ft_z', 'Force verticale Fz (N)', C['blue'], 'Fp_z', 'Fz pression'),
284     ('Ft_y', 'Force latérale Fy (N)', C['red'], None, None),
285     ('disp_z', 'Déplacement z Heave (m)', C['green'], None, None),
286     ('omega_y', 'Vitesse angulaire y (rad/s)', C['orange'], None, None),
287     ('meta_x', 'Hauteur métacentrique GM_x (m)', C['purple'], None, None),
288     ('meta_y', 'Hauteur métacentrique GM_y (m)', C['gray'], None, None),
289 ]
290 positions = [(0,0),(0,1),(1,0),(1,1),(2,0),(2,1)]
291
292 for (col, titre, couleur, col2, lbl2), (r, c) in zip(donnees_ts, positions):
293     ax = fig.add_subplot(gs[r, c])
294     ax.plot(t, df[col].values, color=couleur, lw=1.3, label=titre.split('(')[0].strip())
295     if col2:
296         ax.plot(t, df[col2].values, color=C['green'], lw=0.8, alpha=0.6, label=lbl2)
297     ax.axvline(T_START_REGIME, color='k', ls='--', lw=0.8,
298               label=f't_régime={T_START_REGIME}s')
299     ax.set_title(titre); ax.set_xlabel('Temps (s)')
300     ax.legend(fontsize=8); ax.grid(True, alpha=0.3)
301
302 plt.savefig('fig1_vue_ensemble.png', dpi=150, bbox_inches='tight')
303 plt.show()
304 plt.figure()
305
306 plt.plot(t2[10:], FT[10:, 2], label='OpenFOAM')
307 plt.plot(df['t'][10:], df['Ft_z'][10:], label='Flow3D')
308
309 plt.xlabel('Temps [s]')
310 plt.ylabel('Forces verticales F_z [N]')
311 plt.title('Comparaison des forces totales verticales entre OpenFOAM et Flow3D')
312 plt.legend()
313 plt.show()

```

Listing 3 : Calcule I_{wl} : le moment d'inertie de la surface de flottaison a partir de la vraie section du maillage au plan $Z = z_{wl}$

```

1 import trimesh
2 import numpy as np
3 from shapely.geometry import LineString, MultiPoint
4 from shapely.ops import polygonize, unary_union
5
6 FILE = "/home/uneananash/bateau2.stl"
7 Z_WL = -1.0      # plan de flottaison réel
8 AX_LONG = 0      # X = longitudinal
9 AX_TRAN = 1      # Y = transversal (bâbord/tribord)
10 AX_VERT = 2      # Z = vertical
11
12 mesh = trimesh.load(FILE)
13 if not isinstance(mesh, trimesh.Trimesh):
14     mesh = trimesh.util.concatenate(list(mesh.geometry.values()))
15
16 v = mesh.vertices
17 print(f"Bounds X[{v[:,0].min():.2f},{v[:,0].max():.2f}] "
18       f" Y[{v[:,1].min():.2f},{v[:,1].max():.2f}] "
19       f" Z[{v[:,2].min():.2f},{v[:,2].max():.2f}] ")
20
21 # Section au plan Z = Z_WL contour (X, Y)
22 segs = []
23 for f in mesh.faces:
24     tri = mesh.vertices[f]
25     pts = []
26     for k in range(3):
27         a, b = tri[k], tri[(k+1)%3]
28         da, db = a[AX_VERT]-Z_WL, b[AX_VERT]-Z_WL
29         if da*db <= 0 and abs(db-da) > 1e-12:
30             t = -da/(db-da)
31             p = a + t*(b-a)
32             pts.append((p[AX_LONG], p[AX_TRAN])) # (X, Y) dans le plan de flottaison
33     if len(pts) == 2:
34         segs.append(pts)
35
36 print(f"\n{len(segs)} segments dans le plan Z={Z_WL}")
37
38 if not segs:
39     print("Aucun segment vérifiez Z_WL ou les axes")
40 else:
41     lines = [LineString(s) for s in segs]
42     polys = list(polygonize(lines))
43     all_pts = np.array([p for s in segs for p in s])
44
45     if polys:
46         poly = unary_union(polys)
47     else:
48         poly = MultiPoint(all_pts).convex_hull
49
50     print(f"Aire surface de flottaison A_wl = {poly.area:.4f} m²")
51     coords = np.array(poly.exterior.coords)
52     x_wl = coords[:,0] # longitudinal
53     y_wl = coords[:,1] # transversal
54
55     # Centroïde
56     cx = x_wl.mean()

```

```

57     cy = y_wl.mean()
58
59     # I_wl autour de l'axe longitudinal (pour roulis)
60     # =  $\int y^2 dA$  sur la surface de flottaison
61     # Approximation par la formule de Shoelace étendue (polygone)
62     n = len(coords) - 1
63     Iy = 0.0
64     Ix = 0.0
65     A_check = 0.0
66     for i in range(n):
67         x0,y0 = coords[i]
68         x1,y1 = coords[i+1]
69         cross = x0*y1 - x1*y0
70         Iy += cross * (y0**2 + y0*y1 + y1**2)    # I autour de x=0
71         Ix += cross * (x0**2 + x0*x1 + x1**2)    # I autour de y=0
72         A_check += cross
73
74     A_check = abs(A_check)/2
75     Iy = abs(Iy)/12    # I brut autour de y=0
76     Ix = abs(Ix)/12    # I brut autour de x=0
77
78     # Théorème de Steiner I autour du centroïde
79     Iy_c = Iy - A_check * cy**2    # I_wl pour roulis (axe longitudinal centré)
80     Ix_c = Ix - A_check * cx**2    # I_wl pour tangage
81
82     print(f"\nCentroïde surface de flottaison : X={cx:.3f} m, Y={cy:.3f} m")
83     print(f"Aire vérification : {A_check:.4f} m²")
84     print(f"\nMoment d'inertie surface de flottaison :")
85     print(f"    I_wl (roulis, axe long. centré) = {Iy_c:.4f} m")
86     print(f"    I_wl (tangage, axe tran. centré) = {Ix_c:.4f} m")
87
88     # BM à l'échelle modèle puis réel
89     Vi_modele = 88.0    # m³ à l'échelle modèle (E2)
90     Vi_reel = Vi_modele / 8    # m³
91     BM_modele = Iy_c / Vi_modele
92     BM_reel = (Iy_c * 0.5**2) / Vi_reel    #  $I_{wlE2} / (VE2) = I_{wl} / (VE)$ 
93
94     print(f"\nAvec Vi_modele={Vi_modele} m³, Vi_reel={Vi_reel} m³ :")
95     print(f"    BM (modèle) = {Iy_c:.4f}/{Vi_modele} = {BM_modele:.4f} m")
96     print(f"    BM (réel) = {BM_reel:.4f} m")

```

Listing 4 : Hull Added Mass Inertia Calculator Clip exact des triangles dans chaque tranche $B(x)$, $T(x)$, $A(x)$ propres

```

1
2 import trimesh
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from scipy.ndimage import gaussian_filter1d
6 from shapely.geometry import MultiPoint
7
8 #
9 FILE = "/home/uneananash/OpenFOAM/uneananash-12/run/battt.stl"
10 RHO = 1025.0
11 NX = 80
12 SIGMA = 3
13
14 DRAFT_IMPOSED = None
15 TARGET_DISPLACED_VOLUME = 64
16 #

```

```

17
18 print("Chargement :", FILE)
19 mesh = trimesh.load(FILE)
20 if not isinstance(mesh, trimesh.Trimesh):
21     mesh = trimesh.util.concatenate(list(mesh.geometry.values()))
22
23 verts = mesh.vertices
24 print(f" Vertices:{len(verts)} Faces:{len(mesh.faces)}")
25 print(f" X[{verts[:,0].min():.2f},{verts[:,0].max():.2f}]")
26     f" Y[{verts[:,1].min():.2f},{verts[:,1].max():.2f}]")
27     f" Z[{verts[:,2].min():.2f},{verts[:,2].max():.2f}]")
28
29 extents = verts.max(axis=0) - verts.min(axis=0)
30 axis_order = np.argsort(extents)
31 ax_vert = int(axis_order[0])
32 ax_tran = int(axis_order[1])
33 ax_long = int(axis_order[2])
34 names = ["X", "Y", "Z"]
35 print(f" long={names[ax_long]} tran={names[ax_tran]} vert={names[ax_vert]}")
36
37 v_keel = verts[:,ax_vert].min()
38 v_deck = verts[:,ax_vert].max()
39 L_min = verts[:,ax_long].min()
40 L_max = verts[:,ax_long].max()
41
42
43 # Clip d'un triangle par un plan scalaire
44 def clip_tri_plane(tri, ax, val, keep_below=True):
45     """
46     Clip triangle (3E3) par le plan coord[ax]=val.
47     keep_below=True garde la partie coord[ax] <= val
48     Retourne liste de triangles (chacun shape 3E3).
49     """
50     pts = list(tri)
51     if keep_below:
52         inside = [p[ax] <= val for p in pts]
53     else:
54         inside = [p[ax] >= val for p in pts]
55
56     n = sum(inside)
57     if n == 0:
58         return []
59     if n == 3:
60         return [tri]
61
62     def interp(a, b):
63         t = (val - a[ax]) / (b[ax] - a[ax])
64         return a + t * (b - a)
65
66     if n == 1:
67         idx = inside.index(True)
68         a = pts[idx]
69         b = interp(pts[idx], pts[(idx+1)%3])
70         c = interp(pts[idx], pts[(idx+2)%3])
71         return [np.array([a, b, c])]
72
73     # n == 2
74     idx_out = inside.index(False)
75     out_pt = pts[idx_out]
76     p1 = pts[(idx_out+1)%3]
77     p2 = pts[(idx_out+2)%3]

```

```

78     i1 = interp(out_pt, p1)
79     i2 = interp(out_pt, p2)
80     return [np.array([p1, p2, i1]), np.array([p2, i2, i1])]
81
82
83 def clip_mesh_slab(mesh, ax, x_lo, x_hi, ax_vert, z_wl):
84     """
85     Pour chaque face du maillage :
86     1. clip entre x_lo et x_hi sur ax_long
87     2. clip sous z_wl sur ax_vert
88     Retourne tous les points résultants (NE3).
89     """
90     pts_out = []
91     for f in mesh.faces:
92         tri = mesh.vertices[f]          # shape (3,3)
93
94         # clip entre x_lo et x_hi
95         tris = clip_tri_plane(tri, ax, x_lo, keep_below=False)
96         tris2 = []
97         for t in tris:
98             tris2.extend(clip_tri_plane(t, ax, x_hi, keep_below=True))
99
100        # clip sous la flottaison
101        for t in tris2:
102            clipped = clip_tri_plane(t, ax_vert, z_wl, keep_below=True)
103            for ct in clipped:
104                pts_out.append(ct)      # chaque ct est (3,3)
105
106        if not pts_out:
107            return None
108        return np.vstack(pts_out)      # (N*3, 3) sommets des triangles clipés
109
110
111 # Volume immergé (divergence)
112 def volume_below(mesh, ax, z_wl):
113     total = 0.0
114     for f in mesh.faces:
115         tri = mesh.vertices[f]
116         for ct in clip_tri_plane(tri, ax, z_wl, keep_below=True):
117             a, b, c = ct[0], ct[1], ct[2]
118             total += np.dot(a, np.cross(b, c))
119     return abs(total) / 6.0
120
121
122 # Draft
123 if DRAFT_IMPOSED is not None:
124     z_wl = v_keel + DRAFT_IMPOSED
125     vol = volume_below(mesh, ax_vert, z_wl)
126     draft = DRAFT_IMPOSED
127 else:
128     print(f"\nRecherche draft V={TARGET_DISPLACED_VOLUME} m³ ...")
129     lo, hi = v_keel, v_deck
130     for _ in range(50):
131         mid = 0.5*(lo+hi)
132         if volume_below(mesh, ax_vert, mid) < TARGET_DISPLACED_VOLUME:
133             lo = mid
134         else:
135             hi = mid
136     z_wl = 0.5*(lo+hi)
137     vol = volume_below(mesh, ax_vert, z_wl)
138     draft = z_wl - v_keel

```

```

139     print(f"    {names[ax_vert]}_wl={z_wl:.4f}    draft={draft:.4f} m    V={vol:.4f} m³")
140
141
142 # Strip theory (clip exact par tranche)
143 print(f"\nStrip theory ({NX} tranches, clip exact) ...")
144
145 x_bins = np.linspace(L_min, L_max, NX+1)
146 B_list, T_list, A_list, xm_list = [], [], [], []
147
148 for i in range(NX):
149     x_lo, x_hi = x_bins[i], x_bins[i+1]
150     pts = clip_mesh_slab(mesh, ax_long, x_lo, x_hi, ax_vert, z_wl)
151
152     if pts is None or len(pts) < 6:
153         continue
154
155     y_sec = pts[:, ax_tran]
156     z_sec = pts[:, ax_vert]
157
158     b = y_sec.max() - y_sec.min()
159     t = z_wl - z_sec.min()
160
161     if b < 1e-4 or t < 1e-4:
162         continue
163
164     try:
165         hull = MultiPoint(np.column_stack([y_sec, z_sec])).convex_hull
166         area_sec = hull.area
167     except Exception:
168         area_sec = 0.5 * b * t
169
170     B_list.append(b); T_list.append(t)
171     A_list.append(area_sec); xm_list.append(0.5*(x_lo+x_hi))
172
173 print(f"    {len(B_list)}/{NX} tranches valides")
174
175 if len(B_list) < 3:
176     raise RuntimeError("Trop peu de tranches valides.")
177
178 B = gaussian_filter1d(np.array(B_list), SIGMA)
179 T = gaussian_filter1d(np.array(T_list), SIGMA)
180 A_sec = gaussian_filter1d(np.array(A_list), SIGMA)
181 x_mid = np.array(xm_list)
182
183 print(f"    B=[{B.min():.3f},{B.max():.3f}] m    T=[{T.min():.3f},{T.max():.3f}] m")
184
185
186 # Masses ajoutées
187 a33 = RHO * (np.pi/2.0) * (B/2.0)**2
188 a22 = RHO * (np.pi/2.0) * T**2
189 a44 = RHO * (np.pi/8.0) * (B/2.0)**2 * T**2 / 4.0
190
191 integ = np.trapezoid
192 m33 = integ(a33, x_mid)
193 m22 = integ(a22, x_mid)
194 m44 = integ(a44, x_mid)
195 V_strip = integ(A_sec, x_mid)
196 m11 = 0.07 * RHO * V_strip
197
198 x_cg = integ(a33*x_mid, x_mid) / (m33+1e-12)
199 I55 = integ(a33*(x_mid-x_cg)**2, x_mid)

```

```

200 I44 = m44
201
202
203 # Résultats
204 sep = "="*54
205 print(f"\n{sep}\n  RÉSULTATS\n{sep}")
206 print(f"   Draft                : {draft:.4f} m")
207 print(f"   {names[ax_vert]}_flottaison      : {z_wl:.4f} m")
208 print(f"   Volume (divergence)      : {vol:.4f} m³")
209 print(f"   Volume (strip)          : {V_strip:.4f} m³   écart {abs(vol-V_strip)/vol*100:.1f}%")
210 print(f"   Masse déplacée          : {RH0*vol:.1f} kg\n")
211 print("   Masses ajoutées (Lewis strip theory) :")
212 print(f"       m11 surge : {m11:.2f} kg")
213 print(f"       m22 sway  : {m22:.2f} kg")
214 print(f"       m33 heave : {m33:.2f} kg")
215 print(f"       m44 roll  : {m44:.2f} kgûmš\n")
216 print("   Inerties quadratiques :")
217 print(f"       I55 tangage: {I55:.2f} kgûmš")
218 print(f"       I44 roulis : {I44:.2f} kgûmš")
219 print(sep)
220
221
222 # Graphiques
223 fig, axes = plt.subplots(3, 1, figsize=(11, 9), sharex=True)
224
225 axes[0].plot(x_mid, B, "steelblue", lw=2, label=f"B({names[ax_long]}) [m]")
226 axes[0].set_ylabel("Largeur B [m]")
227 axes[0].set_title("Géométrie du carène immergé clip exact par tranche")
228 axes[0].legend(); axes[0].grid(True)
229
230 axes[1].plot(x_mid, T, "darkorange", lw=2, label=f"T({names[ax_long]}) [m]")
231 axes[1].set_ylabel("Tirant T [m]")
232 axes[1].legend(); axes[1].grid(True)
233
234 axes[2].plot(x_mid, a33, "green", lw=2, label="a heave [kg/m]")
235 axes[2].plot(x_mid, a22, "red", lw=2, label="a sway [kg/m]")
236 axes[2].set_ylabel("Masse ajoutée [kg/m]")
237 axes[2].set_xlabel(f"{names[ax_long]} [m]")
238 axes[2].legend(); axes[2].grid(True)
239
240 plt.suptitle(
241     f"Draft={draft:.3f}m | m33={m33:.0f}kg | m22={m22:.0f}kg | I55={I55:.0f}kgûmš",
242     fontsize=10
243 )
244 plt.tight_layout()
245 out_fig = FILE.replace(".stl", "_analysis.png")
246 plt.savefig(out_fig, dpi=150)
247 print(f"\nGraphique {out_fig}")
248 plt.show()

```